

Notes of Computer and Information Sciences (Master Level)

From practice to mastering (Semester one)

Zhao, Wei

Department/Role

Organization/Company

newfishjerry@gmail.com

June 22, 2026

Contents

1	Symbols	1
2	Deep Learning(COMP838)	3
2.1	Artificial Neural Networks	3
2.1.1	Feedforward Neural Networks	4
2.1.2	Three Datasets	4
2.1.3	Three Functional Operations	4
2.1.4	Activation Function	5
2.1.5	Loss Functions	5
2.1.6	Backpropagation	5
2.2	Well-known Models	6
2.3	Convolutional Neural Network (CNN)	6
2.3.1	AlexNet	7
2.3.2	R-CNN (Region-based CNN)	7
2.3.3	YOLO (You Only Look Once)	7
2.3.4	SSD(Single Shot MultiBox Detector)	8
2.4	Recurrent Neural Network (RNN)	8
2.4.1	LSTM	8
2.4.2	GRU	8
2.5	GPT (Generative Pre-trained Transformer)	9
2.6	Reinforcement Learning	10
2.7	Transfer Learning	10
2.7.1	How it Works (The Process)	10
2.7.2	Summary	10
2.8	Ensemble Learning	11
2.9	Generative AI	11
2.9.1	Transformer-Based Models (LLMs)	11
2.10	GANs (Generative Adversarial Networks)	11
2.11	Diffusion Models	11
2.12	VAEs (Variational Autoencoders)	12
2.12.1	Important Correction First	12
2.13	Confusion Matrix	12
2.13.1	AUC-ROC	13
3	Data Mining and Machine Learning(COMP809)	15
3.1	Setup Environment	15
3.1.1	Jupyter Notebook in VSC (Windows)	15
3.1.2	Jupyter Notebook without VSC (Windows)	15
3.1.3	Jupyter Notebook On CachyOS (Linux Arch distribution)	15
3.2	Introduction	15
3.3	Data Mining vs Statistics	16

3.4	Types of Variables	16
3.5	Support Vector Machine (SVM)	17
3.6	Data Exploration	17
3.6.1	Finding Structure	18
3.6.2	Overfitting: signal vs. noise	18
3.6.3	Regularization	18
3.6.4	Modelling Structure	19
3.7	Regression Models	19
3.7.1	Simple Linear Regression	20
3.7.2	Parameter Estimation least squares	20
3.7.3	Model Assumptions	20
3.7.4	Robust Linear Regression	21
3.7.5	Comparing Models	22
3.7.6	Multiple Linear Regression	22
3.8	Missing Values in Regression	22
3.8.1	Imputation Methods	23
3.9	Multicollinearity	23
3.9.1	Variance Inflation Factor (VIF)	23
3.10	Curse of Dimensionality	24
3.11	Feature Selection	24
3.11.1	Pearson's Correlation	25
3.12	Feature Extraction	26
3.12.1	PCA (Principal Component Analysis)	26
3.12.2	Biplot	26
3.13	Non-Numeric Input Coding	27
3.13.1	Shrinkage methods	27
3.14	Generalized Linear Models (GLMs)	29
3.14.1	The Three Core Components	29
3.14.2	Overdispersion	30
3.14.3	Data Preprocessing	30
3.15	Unsupervised and Supervised	31
3.16	Clustering Analysis	31
3.16.1	Distance Measures	31
3.16.2	Methods of Cluster Analysis	32
3.16.3	Summary	34
3.17	Decision Tree	34
3.17.1	Supervised and Unsupervised Learning	34
3.18	Naive Bays	35
3.18.1	Properties of Conditional Probability	35
3.18.2	Naive Bayes	35
3.18.3	Bayes Classifiers	36
3.18.4	General Characteristics of Naïve Bayes	36
3.19	Practical Issues in Classification	36
3.20	Methods for Performance Evaluation	36
3.21	Methods for Model Comparison	36
4	Statistical Programming for Data Sciences(COMP828)	37
4.1	Descriptive Statistics	37
4.2	Plots	37
4.3	mode function in R	38
4.4	Factors	38
4.4.1	When You MUST Transform	38
4.4.2	How to Do It in R	38
4.4.3	Why It Matters for Models and Plots	39

4.4.4	The "Binning" Technique	39
5	Video and Image Processing(COMP820)	41
5.1	Assignment	41
5.1.1	Assignment 1 and 2 topic context suggestions: Privacy-preserving human activity recognition systems	41
5.1.2	How to do it	42
5.2	Self-directed Learning: Matlab Resources for Image Enhancements	42
5.3	Homework and self-paced learning courseware	43
5.3.1	MatLab Image Processing Workflow	43
5.3.2	Conclusion	47
5.3.3	Demo	48
5.4	Lecture 3	48
5.5	Other Learning Resources	49
6	Mathamatics	51
6.1	Norm	51
6.2	Harmonic Mean	51
6.3	Marginal Probability	51
6.4	Conditional Probability	52
6.5	Distributions	52
6.5.1	Bernoulli Distribution	52
6.5.2	Binomial Distribution	53
6.5.3	Poisson Distribution	53
6.5.4	Normal Distribution	54
6.5.5	Exponential Distribution	54
6.6	Central Limit Theorem	55
6.7	Pearson chi-square	55
6.7.1	Step 1/4: Gethering Data	55
6.7.2	Step 2/4: Calculate Expected Counts E	55
6.7.3	Step 3/4: Calculate Pearson chi-square result	56
6.7.4	Step 4/4: Calculate Dispersion Parameter and Interpret	56
6.8	Covariance	56
6.9	Softmax	57
6.9.1	What softmax does (intuition)	57
6.9.2	Key properties	57
6.9.3	Why exponentials? (important insight)	58
6.9.4	Numerical stability trick (very important in practice)	58

Chapter 1

Symbols

Term	Description
XOR	Exclusive OR logical operation
Confidence level	Probability that a confidence interval contains the true parameter
$\tanh$	Hyperbolic tangent function: $\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$
OLS	Ordinary Least Squares regression method
Confidence intervals	(range where the parameter is expected to lie)
Variance	$\sigma^2 = \frac{1}{n} \sum_i (x_i - \bar{x})^2$
Covariance	variance between two different data
Standard deviation	Square root of variance: $\sigma = \sqrt{\sigma^2}$
Outlier	Observation that deviates significantly from other data points
Pearson Correlation Coefficient	: $r = \frac{\text{cov}(X,Y)}{\sigma_X \sigma_Y}$
residual	:
Multicollinearity	
Dummy variable trap	
VIF	Variance Inflation Factor
NLP	Natural Language Processing
Confusion Matrix	
Curse of Dimensionality	
PCA	Principal Component Analysis
CV(Coefficient of Variation)	$CV = \frac{\sigma}{\mu}$ ()
RSS	Residual Sum of Squares OR sum of squared residuals.
Regularization	This technique is used to prevent overfitting by adding a penalty term to the loss fu
Normalization	ypically refers to scaling data to a fixed range, most commonly between 0 and 1.
Standardization	Refers to rescaling data so that it has a mean of 0 and a standard deviation of 1 (of
L_n norm	L_n
Null Hypothesis	
Alternative Hypothesis	/
Significance Level	
Harmonic Mean	
Marginal Probability	
Joint probability	
Conditional probability	
iid	independent and identically distributed ()
Dummy Variable	
GLM	Generalized Linear Models
odds	$\frac{p}{1-p}$

log-odds	$\ln(\frac{p}{1-p})$
$Y \sim \text{Binomial}()$	read as: Y is distributed as Binomial
Pearson Chi-Square	
logistic model	
saturated model	
Residual Deviance	
Dispersion Parameter	
Over-dispersion	
HAR	human activity recognition
K-Means	K-
EDA	Exploratory Data Analysis
Imputation Methods	
regression imputation	
Maximum A Posteriori (MAP)	
MLE	Maximum Likelihood Estimation
Prior probability $P(H)$	EH
Prior probability $P(H E)$	
Likelihood $P(EH)$	
Evidence/Marginal Likelihood $P(E)$	/
Cross-Entropy Loss Function	
WCSS	Within-Cluster Sum of Square
Naive Bays	

Chapter 2

Deep Learning(COMP838)

2.1 Artificial Neural Networks

Table 2.1: Machine Learning(ML) vs Deep Learning(DL)

Machine Learning	Deep Learning
Good results with small data sets	Requires very large data sets
Quick to train a model	Computationally intensive
Need to try different features and classifier to achieve best results	Learns features and classifiers automatically
Accuracy plateaus	Accuracy is unlimited

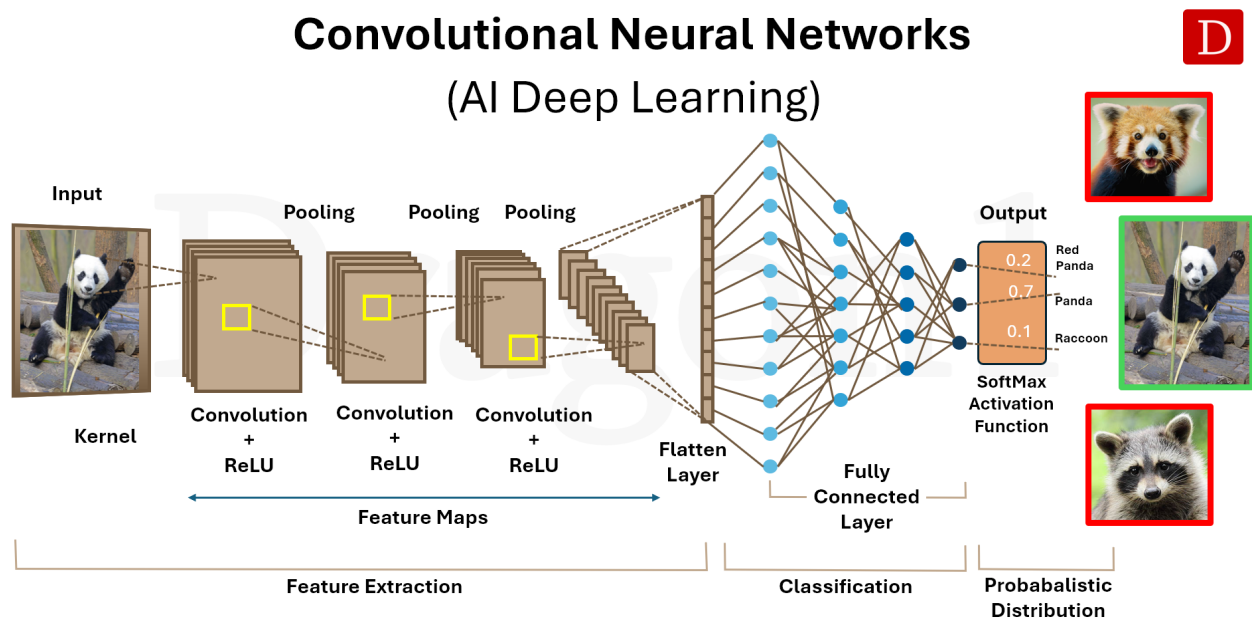


Figure 2.1: Architecture of Convolutional Neural Network (© Copyright 2025, www.dragon1.com)

2.1.1 Feedforward Neural Networks

- Multilayer networks are capable of performing any linear or nonlinear computations, they can approximate any reasonable function arbitrarily well.
- Networks are also sensitive to the number of neurons in their hidden layers.
- Too few neurons can lead to underfitting.
- Too many neurons can contribute to overfitting, in which all training points are well fitted, but the fitting curve oscillates wildly between these points.

2.1.2 Three Datasets

1. **Training dataset:** A set of examples are used to fit the parameters of the model. The dataset often consists of pairs of an input vector and the corresponding output vector. (70%)
2. **Test dataset:** A set of samples are used only to assess the performance (i.e., generalization) of a fully specified classifier.(15%)
3. **Validation dataset:** A dataset of samples are used to tune the hyperparameters (i.e., the architecture) of a classifier.(15%)

A dataset can be repeatedly split into a training dataset and a validation dataset, we call this **Cross Validation**.

- Deep learning is a branch of machine learning that trains computers to do what comes naturally to humans: Learn from experience.
- Machine learning algorithms use computational methods to learn information directly from data without relying on a predetermined equation as a model.
- Deep learning is especially suitable for computer vision, which is important for solving problems such as face recognition, motion detection, and many advanced driver assistance technologies.

Deep Neural Networks(DNNs)

- A deep neural network combines multiple nonlinear processing layers, using simple element operating in parallel and inspired by biological nervous systems.
- A deep neural network consists of an input layer, several hidden layers, and an output layer.
- The layers are interconnected via nodes, or neurons, with each hidden layer using the output of previous layer as its input.

2.1.3 Three Functional Operations

1. Vector weight function: Form the product $\mathbf{W} \cdot \mathbf{p}$
2. Vector input function: Form the net input $\mathbf{n} = \mathbf{W} \cdot \mathbf{p} + b$
3. Vector transfer function: Produce the scalar output $\mathbf{a} = f(\mathbf{W} \cdot \mathbf{p} + b)$

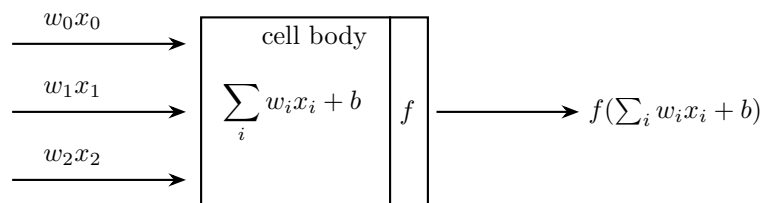


Figure 2.2: Neuron and its Signals

$$\mathbf{Y} = \sigma(w_1 \mathbf{X}_1 + w_2 \mathbf{X}_2 + \cdots + w_n \mathbf{X}_n + \mathbf{b}) \quad (2.1)$$

Activation: This is the math used inside each neuron. *ReLU* is the modern standard and usually works best. *Sigmoid* and *Tanh* are older methods that can be slower.

Learning Rate (η): How fast the model adjusts.

- If it's too high (e.g. 10), the model might "jump" over the solution and go crazy.
- If it's too low (e.g. 0.00001), it will take forever to learn. **0.03** is a good starting point.

Regularization: This helps prevent the model from being too "twitchy" or over-complex.

2.1.4 Activation Function

σ is the Activation function. Weights ($\mathbf{w}$) are per connection (each line has its own weight). Biases ($\mathbf{b}$) are per neuron (each circle has its own bias).

$$y = \sigma(\mathbf{w} \cdot x + \mathbf{b}) \quad (2.2)$$

Table 2.2: Common Used Activation Functions

Name	Function	Range
ReLU	$\sigma(x) = \max(0, x)$	$[0, \infty]$
tanh	$\sigma(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$	$[-1, 1]$
Sigmoid ^a	$\sigma(x) = \frac{1}{1 + e^{-x}}$	$[0, 1]$

^a Sigmoid is actually inverse function of Logit function.

2.1.5 Loss Functions

Loss functions measure how far a models prediction is from the true value. Different tasks use different loss functions. Below are the most commonly used ones.

Table 2.3: Classification Loss Functions

FUNCTION	EQUATION	DESCRIPTION
Binary Cross-Entropy ¹	$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$	logistic regression binary neural networks
Categorical Cross-Entropy ²	$L = -\sum_1^C y_i \log(\hat{y}_i)$	C = number of classes softmax output layer
Sparse Categorical Cross-Entropy		

¹ Used for binary classification.

² Used for multi-class classification.

2.1.6 Backpropagation

Backpropagation (backward propagation of errors) is an algorithm used to train neural networks. It calculates how much each weight in the network contributed to the prediction error and then adjusts those weights to reduce the error. In simple terms, it answers the question: Which weights in the network should change, and by how much, to make the prediction better?

Taylor Expansion

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!} (x - a)^n \quad (2.3)$$

why old weight subtract derivative of weight can guarantee overall loss descent?

For a small change in weight Δw , the change in loss L can be approximated (2.3) as:

$$L(w + \Delta w) \approx L(w) + \frac{\partial L}{\partial w} \Delta w$$

If we define our change Δw as the negative derivative $-\eta \frac{\partial L}{\partial w}$, and substitute it back into the equation:

$$\begin{aligned} L(w_{new}) &\approx L(w) + \frac{\partial L}{\partial w} \left(-\eta \frac{\partial L}{\partial w} \right) \\ &\approx L(w) - \eta \left(\frac{\partial L}{\partial w} \right)^2 \end{aligned}$$

Since any real number squared is always non-negative, and the learning rate η is positive, we are effectively subtracting a positive value from the current loss. Thus: $L(w_{new}) \leq L(w_{old})$

2.2 Well-known Models

Table 2.4: Well-Known Models and Their Time

CNN	AlexNet(Deep Learning Revolution)	2012
	R-CNN	2013
	ResNet	2015
	GoogLeNet	2015
	SSD	2015
	YOLO	2016
RNN	LSTM	1997
	GRU	2014
GAN(2014)		
Transformer(2017)	GPT	2018

2.3 Convolutional Neural Network (CNN)

CNN has three operations, they are:

- **Convolution** puts the input through a set of convolutional filters.
- **Pooling** simplifies the output through nonlinear downsampling.
- **ReLU (Rectified Linear Unit)** allows for fast and effective training by mapping negative values to zero and maintaining positive ones.

A fully connected layer (**FC**) outputs a vector of k dimensions where k is the number of classes that the net is able to predict. The vector contains the probabilities for each class of any images being classified. The final layer of the CNN architecture uses a softmax (section 6.9) function to provide the classification output.

ConvNets are inspired from the biological structure of a visual cortex, which contains arrangements of simple and complex cells. These cells are activated based on the subregions of a visual field, i.e., **receptive field**. A ConvNet reduces the number of parameters with the number of connections, shared weights, and downsampling. A ConvNet consists of multiple layers, such as convolutional layers, max pooling or average pooling layers, and fully connected layers.

2.3.1 AlexNet

AlexNet is a deep convolutional neural network (CNN) that revolutionized computer vision in 2012.

- **Before 2012:**
 - Traditional computer vision relied on hand-crafted features.
 - Deep learning wasn't dominant yet.
- **AlexNet:**
 - Reduced ImageNet error rate from $\sim 26\%$ to $\sim 15\%$
 - Proved deep CNNs + GPUs work extremely well
 - Triggered the deep learning revolution

2.3.2 R-CNN (Region-based CNN)

R-CNN (Regions with Convolutional Neural Networks) is the grandfather of modern object detection. It is based on CNNs, but unlike the "one-stage" models (YOLO/SSD), it is a two-stage model: it finds the "where" first, and then asks the "what" later. It is a two-stage detection algorithm:

1. The first stage identifies a subset of regions in an image that might contain an object.
2. The second stage classifies the object in each region.

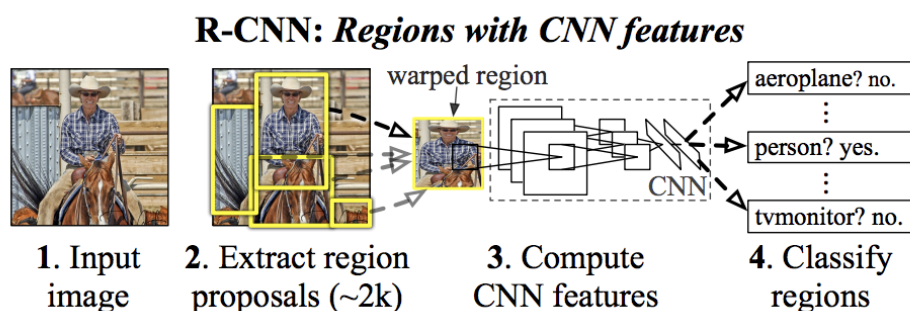


Figure 2.3: The R-CNN Architecture, featuring the steps of taking in an input image, extracting region proposals, computing CNN features, and classifying regions.

Why it was a revolution?

Before R-CNN came out in 2014, object detection was mostly based on very simple mathematical templates. R-CNN proved that:

- Deep Learning (CNNs) could be used to find objects, not just classify them.
- Transfer Learning worked you could take a CNN trained on ImageNet and "fine-tune" it for detection.

The reason we eventually moved to YOLO and SSD is that R-CNN was incredibly inefficient:

- **2,000 passes:** It had to run a CNN 2,000 times for every single image.
- **Time:** It took about 47 seconds to process one image on a high-end GPU. You couldn't use it for video or self-driving cars.
- **Storage:** You had to save the features of those 2,000 boxes to a hard drive during training, which took up hundreds of gigabytes.

2.3.3 YOLO (You Only Look Once)

YOLO is a real-time object detection algorithm that detects objects in an image in a single pass rather than scanning multiple times. The name (You Only Look Once) reflects the fact that the model processes image

Table 2.5: R-CNN and its Evolutions

Model	Key Improvements	Speed
R-CNN(2014)	first to use CNNs for detection ^a	47sec/image
Fast R-CNN(2015)	processes the whole image once before cropping features	2sec/image
Faster R-CNN(2016)	replaced "Selective Search" with a dedicated AI network.	0.2sec/image

^a CNN(before R-CNN) could tell you what was in the image and its probability (92% of Dog, 4% of Cat, 1% of Car), but it had no idea where it was or how many of them. If there were three dogs in the photo, CNN would still just say Dog.

once to detect everything. Traditional object detectors, like R-CNN (Section 2.3.2), use two stages:

1. Generate region proposals for possible objects;
2. Classify objects within those regions.

YOLO treats detection as a single regression problem: input an image then output a bounding boxes and class probabilities for all objects.

2.3.4 SSD(Single Shot MultiBox Detector)

Released shortly after YOLO by researchers at Google/UNC, SSD was designed to fix one of YOLOs early weaknesses: small objects.

2.4 Recurrent Neural Network (RNN)

In deep learning, Long Short-Term Memory (**LSTM**) and Gated Recurrent Unit (**GRU**) are special types of Recurrent Neural Networks (**RNNs**) designed to handle long-term dependencies in sequential data (like text, speech, or time series).

2.4.1 LSTM

Traditional **RNNs** suffer from the *vanishing gradient problem*, meaning they struggle to remember information from far back in a sequence. **LSTM** introduces a *cell state (memory)* and uses gates to control information flow to solve this. It has three gates:

- **Forget Gate:** Decides what information to remove from memory.
- **Input Gate:** Decides what new information to store.
- **Output Gate:** Decides what part of memory to output.

2.4.2 GRU

GRU is a simplified version of LSTM. It aims to:

- Be computationally simpler
- Train faster
- Use fewer parameters
- Perform similarly to LSTM in many tasks

Two Gates in GRU:

1. Update Gate: Controls how much past information to keep.
2. Reset Gate: Controls how much past information to forget.

Table 2.6: LSTM vs GRU

Feature	LSTM	GRU
Gates	3	2
Cell state	Yes	No
Parameters	More	Fewer
Speed	Slower	Faster
Performance	Strong for long sequence	Comparable in many cases

When to Use Which?

Use LSTM when:

- Very long dependencies matter
- You want more control over memory

Use GRU when:

- You want faster training
- Dataset is smaller
- Simpler model preferred

2.5 GPT (Generative Pre-trained Transformer)

GPT is based on the **Transformer** architecture, not RNN.

Theses about GPT

Original paper: [Attention Is All You Need](#) (introduced Transformer)

First GPT paper: [Improving Language Understanding by Generative Pre-Training](#)

Instead of processing sequentially like **RNN**, **GPT** uses *Self-Attention Mechanism*. Each word looks at all other words in the sentence simultaneously. So instead of using (2.4), it computes **attention weights** between all tokens.

$$h_t = f(x_t, h_{t-1}) \quad (2.4)$$

Table 2.7: Major Differences

Feature	RNN	GPT
Architecture	Recurrent	Transformer
Processing	Sequential	Parallel
Memory	Hidden state	Attention mechanism
Long-range dependency	Weak	Strong
Training speed	Slow	Fast (parallelizable)
Scalability	Limited	Extremely scalable

Intuition Example

Sentence: **The animal didnt cross the street because it was tired.**

RNN: Must carry memory **step-by-step** from beginning to *it*.

GPT: *It* directly attends to *animal* via attention weights. **No need to pass** information through many steps.

Why GPT replaced RNN in NLP(Natural Language Processing)?

- Better at long-distance relationships
- Parallel training on GPUs
- Scales extremely well (billions/trillions of parameters)
- Works for text, image, audio (multimodal)

Simple Summary

RNN = memory chain, passes information forward like a relay race.

GPT = attention graph, connects everything directly.

2.6 Reinforcement Learning

Think of Reinforcement Learning (**RL**) as the way you would train a dog: you don't give the dog a manual on "how to sit," you just give it a treat when it happens to sit, and a stern "no" when it jumps on the couch. In AI terms, it is a type of machine learning where an Agent learns to make decisions by performing actions in an Environment to maximize a Reward. To understand RL, you have to look at the "Loop" that happens millions of times during training:

- **Agent:** The "student" or the AI (e.g., a robot or a character in a video game).
- **Environment:** The world the agent lives in (e.g., a maze or a stock market).
- **Action (A):** What the agent chooses to do (e.g., move left, buy a stock, jump).
- **State (S):** The current situation (e.g., the agent's coordinates in the maze).
- **Reward (R):** The feedback. Positive for a good job, negative for a mistake.

2.7 Transfer Learning

It is a technique where you take a model that has already been trained on a massive dataset (like 1.2 million photos in ImageNet) and "transfer" its knowledge to a new, smaller task. Instead of starting from zero and teaching a model what an "edge," a "circle," or a "texture" is, you start with a model that is already an expert at seeing and just teach it the final details of your specific project. In AI, if a model (like ResNet) already knows how to recognize animals, it will learn to recognize cancerous cells in X-rays much faster than a model that has never seen an image before.

2.7.1 How it Works (The Process)

When you use transfer learning, you typically follow these three steps:

1. **Select a Pre-trained Model:** You download a "Backbone" model that was trained by a big company (Google, Microsoft, Meta) on a huge dataset.
2. **Freeze the Base:** You "lock" the early layers of the network. These layers have already learned general features (edges, shapes, colors) that are useful for any image. You don't want to change these because they are already perfect.
3. **Replace the "Head":** You chop off the very last layer (the part that says "This is a dog") and replace it with your own layer (e.g., "This is a rusty bolt" vs. "This is a clean bolt"). You only train this new part on your small dataset.

2.7.2 Summary

Transfer learning allows you to "stand on the shoulders of giants." You use the billions of dollars spent by tech giants to build your own high-quality AI with very little data.

2.8 Ensemble Learning

Ensemble Learning is the strategy of combining multiple models to solve a single problem. The core philosophy is that "a group of specialists is smarter than a single genius". In an ensemble, you train several "base learners" (which could be SVMs, CNNs, or simple Decision Trees) and then aggregate their predictions. This reduces the risk of making a major error because one model's mistake is often corrected by the others.

2.9 Generative AI

Generative AI (Gen AI) refers to models that learn patterns from data and then generate new content that resembles the training data. Instead of only classifying or predicting (like traditional machine learning), generative models create:

- Text
- Audio
- Video
- Code
- Images
- 3D models

2.9.1 Transformer-Based Models (LLMs)

- They are mainly used for:
- Text generation
- Code generation
- Question answering
- Chatbots
- Translation

Examples of systems:

- ChatGPT
- Gemini
- Claude

Core idea: They predict the next token given previous tokens:

$$P = (x_t | x_1, x_2, \dots, x_{t-1})$$

2.10 GANs (Generative Adversarial Networks)

Introduced by Ian Goodfellow in 2014.

Generator creates fake data

Discriminator tries to detect fake vs real

Used for:

- Image synthesis
- Deepfakes
- Style transfer

2.11 Diffusion Models

Popular in modern image generation.

Examples:

Stable Diffusion

DALL E

Core idea:

Gradually add noise to real images.

Train model to reverse the noise process.

Generate new images by starting from noise and denoising step by step.

2.12 VAEs (Variational Autoencoders)

Architecture:

Encoder compress input to latent vector

Decoder reconstruct from latent space

They are more mathematically structured than GANs but usually produce blurrier outputs. After the input is compressed into a latent vector, what exactly happens?

2.12.1 Important Correction First

In a normal **autoencoder**: $x \rightarrow z \rightarrow \hat{x}$

The encoder outputs a single deterministic vector z . But in a Variational Autoencoder (VAE), the encoder does not output a single latent vector. It outputs two vectors: $\mu(x)$ $\sigma(x)$. These define a distribution:

$$q(z|x) = \mathcal{N}(\mu(x), \sigma^2(x)) \quad (2.5)$$

So the encoder produces a **mean** and **variance**, not just a compressed code. But what Happens After Compression?

1. Sampling (Reparameterization Trick): Instead of directly using μ as latent vector, we sample noise: $\epsilon \sim \mathcal{N}(0, 1)$, then construct latent vector: $z = \mu + \sigma \cdot \epsilon$. This is called the *reparameterization trick*, if sampling directly from $\mathcal{N}(\mu, \sigma^2)$ would break backpropagation, by rewriting it this way, gradients can flow through μ and σ .
2. Decoder Takes z : Now the sampled latent vector z is fed into the decoder network.
3. Two Loss Components: VAE does NOT only minimize reconstruction error.

2.13 Confusion Matrix

A confusion matrix is a table used to evaluate the performance of a classification model. It compares the actual labels with the predicted labels made by the model.

Why it is called **confusion**?

It shows where the model gets confused between classes (e.g., predicting negative when it should be positive).

The **F1 Score** is a metric used to evaluate classification models. It combines **Precision** and **Recall** into a single number. It is especially useful when you want a balance between precision and recall. The F1 score is the harmonic mean (Section 6.2) of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

This means:

- If both precision and recall are high, F1 will be high.
- If either one is low, F1 will drop.

Table 2.8: Confusion Matrix of A Binary Classification Problem (two classes, e.g. positive vs negative)

	Predicted Positive	Predicted Negative	
Actual Positive	True Positive (TP) ¹	False Negative (FN) ⁴	Recall / Sensitivity
Actual Negative	False Positive (FP) ³	True Negative (TN) ²	Specificity
	Precision (PPV)		Accuracy (ACC)

¹ Model correctly predicts the positive class.

² Model correctly predicts the negative class.

³ Model predicts positive but actually negative. Type I error.

⁴ Model predicts negative but actually positive. Type II error.

2.13.1 AUC-ROC

AUC-ROC (Area Under the ROC Curve) is a performance metric used to evaluate classification models, especially binary classifiers. It shows how well a model separates the two classes across different thresholds.

The ROC curve plots:

- **X-axis:** False Positive Rate (FPR) = $FP / (FP + TN)$
- **Y-axis:** True Positive Rate (TPR) = Recall = $TP / (TP + FN)$

AUC is the area under the ROC curve. It summarizes the model's performance into a single number. Interpretation:

- AUC = 1.0 -> Perfect classifier
- AUC = 0.5 -> Random guessing
- AUC < 0.5 -> Worse than random (model is misleading)

Chapter 3

Data Mining and Machine Learning(COMP809)

3.1 Setup Environment

3.1.1 Jupyter Notebook in VSC (Windows)

1. install python: `winget install -id python.Python.3.14 -source winget -e`
2. install necessary packages using pip:
`pip install numpy`
`pip install pandas`
`pip install seaborn`
3. install VSC: `winget install -id TeXstudio.TeXstudio -source winget -e`
4. install VSC plugins: python and jupyter
5. done

3.1.2 Jupyter Notebook without VSC (Windows)

install: `pip install notebook`

and then run: `jupyter notebook`

or run: `C:/Users/Jerry/AppData/Roaming/Python/Python314/Scripts/jupyter.exe notebook`

if something is wrong, just go to python script folder, append folder to PATH

3.1.3 Jupiter Notebook On CachyOS (Linux Arch distribution)

Use python to create a virtual environment, then activate the env. Finally, install jupyter notebook just as 3.1.2 do.

```
python -m venv .env
```

```
source .env/bin/activate
```

done

3.2 Introduction

Data Mining is the process of extracting and discovering patterns in large data sets. It also includes the study and practice of data storage and data manipulation. Two major categories:

- **Description methods:** Find human interpretable patterns that describe the data (correlations, trends, clusters, anomalies etc).

- **Prediction methods:** Use data to predict unknown future values

Machine Learning (ML) is a method of data analysis that automates analytical model building. Machine learning is also often split into:

- **Unsupervised ML** methods learn patterns from untagged data (no response variable). E.g. clustering/PCA/Association Analysis
- **Supervised ML** methods learn patterns from labelled data. E.g. predictive modelling techniques (regression/trees/SVMs etc).

3.3 Data Mining vs Statistics

- **Traditional statistics**
 - first hypothesize, then collect data, then analyze
 - often model-oriented (strong parametric models).
- **Data mining**
 - few if any a priori hypotheses
 - data is usually already collected a priori
 - analysis is typically data-driven not hypothesis-driven
 - Often algorithm-oriented rather than model-oriented

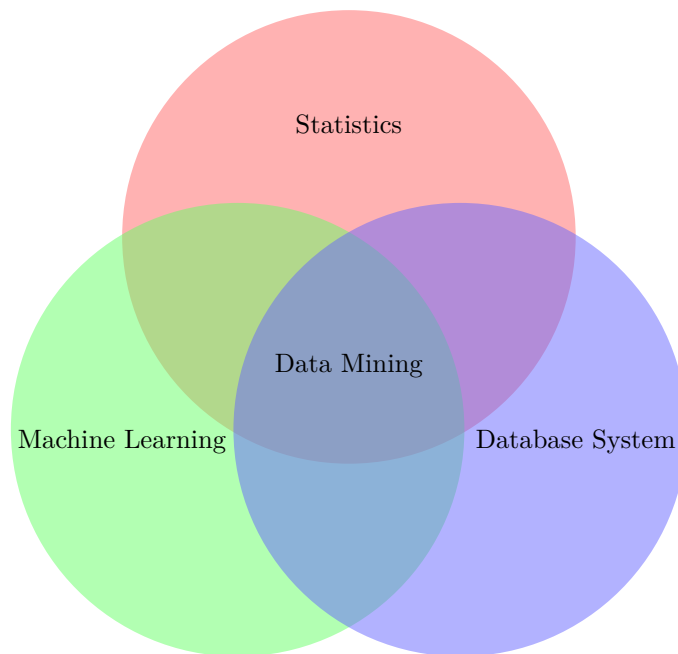


Figure 3.1: Relationship Between Related Subjects

3.4 Types of Variables

Categorical:

- **Nominal:** label that provide only enough information to distinguish one object from another. (ID number, gender)
- **Ordinal:** label that provide enough information to order the objects. (hardness of minerals good, better, best)

Numerical:

Table 3.1: Statistics vs Data mining

Statistics	Data Mining
Low dimensional data (typically < 100 dimensions) 'Noisy' data	High dimensional data (> 100 dimensions) With correct processing, noise does not obscure structure
Little structure in the data. (Structure representable by fairly simple mode)	A great deal of structure in the data. (Structures are generally complicated)
Main problems:	
Distinguishing true structure from noise	Representing the complicated structure

- **Discrete:** can assume only a finite number of real values within a given interval. (number of students in the classroom)
- **Continuous:** can assume an infinite number of real values within a given interval. (height of a person)
- Continuous variables are classified as:
 - **Interval:** numbers in which the differences between values are meaningful. (calendar dates, temperature in Celsius or Fahrenheit)
 - **Ratio:** numbers in which both differences and ratios are meaningful. (temperature in Kelvin, counts, age, length)

3.5 Support Vector Machine (SVM)

An SVM is a powerful supervised learning model used for classification and regression. If a CNN is about "seeing" and an RNN is about "remembering," an SVM is about "finding the best boundary." Imagine you have a plot of land with two types of trees: Oak and Pine. An SVM tries to draw a fence that separates them as widely as possible.

Table 3.2: Comparison Between Support Vector Machines (SVM) and Deep Learning

Feature	SVM	Deep Learning
Data Size	Performs well on small to medium datasets	Requires massive datasets to reach peak accuracy
Interpretability	High; based on geometric margins ¹	Low; often considered a "black box"
Training Speed	Generally fast and mathematically stable	Slow; requires high-end GPUs and tuning
Feature Selection	Uses the "Kernel Trick" for non-linear data	Automates feature extraction through layers
Complexity	Struggles with raw high-res images/video	Excellent for unstructured data (images, text)

¹ Support vectors provide clear indicators of the decision boundary's location.

3.6 Data Exploration

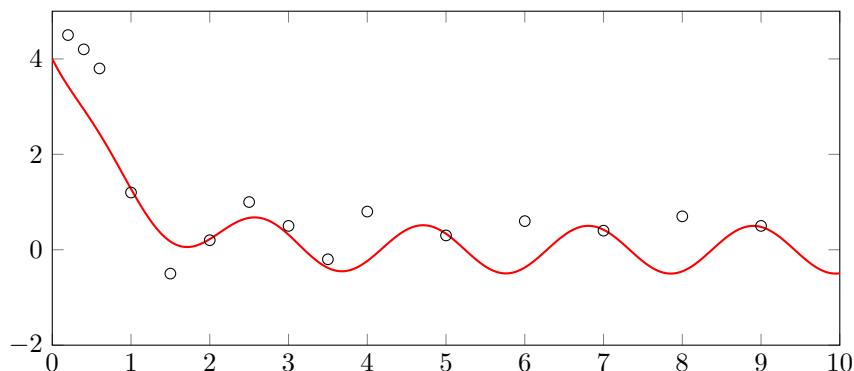
An approach/philosophy for data analysis that employs a variety of techniques (mostly graphical) is Exploratory Data Analysis (EDA), initiated by John Tukey. The focus was on visualization. Clustering and anomaly detection were viewed as exploratory techniques. In data mining, clustering and anomaly detection are major areas of interest, and not thought of as just exploratory. EDA is the first step in any analysis. A preliminary exploration of the data is used to better understand its characteristics.

Key motivations of data exploration include:

- Helping to select the right tool for preprocessing or analysis.

- Making use of humans abilities to recognize patterns.
- We can recognize patterns not captured by data analysis tools.
- Detect anomalies.

3.6.1 Finding Structure



You can think of data as being composed of:

$$\text{DATA} = \text{STRUCTURE} + \text{NOISE}$$

Predictive modelling in DM is essentially minimizing NOISE by finding a sensible structure (pattern).

$$\text{i.e. NOISE} = (\text{DATA} - \text{STRUCTURE})$$

3.6.2 Overfitting: signal vs. noise

In every dataset, there are two types of variation:

1. The Signal
 - consistent and repeatable
 - predictable
 - Interesting
2. The Noise
 - random
 - unpredictable the same noise will not be present in other equivalent datasets
 - not interesting (except perhaps to quantify)
 - superimposed onto the signal

Your models seek to describe only the signal, not the noise. Ability to do this depends on the relative magnitude of these components in data. If the signal is large and the noise small, it is relatively easy to find a good model which makes consistent predictions. But if the noise is large and the signal small, it is more difficult to tease out the underlying signal, making predictive modelling difficult and highly variable.

It is possible to create a model that goes beyond describing the predictable signal, and also describes some of the noise. This is known as **overfitting**. The model then performs poorly when predicting new data, because the variation that it describes does not generalise to other datasets. Constraining a model to reduce overfitting is often called **regularisation**.

3.6.3 Regularization

Regularization is about the failure of statistical learning to adequately predict generalization error, it is used to prevent overfitting. Typical examples of regularization in statistical machine learning include shrinkage methods (Section 3.13.1), e.g. ridge regression, LASSO, bayesian priors. More informations check [cambridgecoding](#).

To understand first sentence, let's break it down into two main parts: **the problem** (predicting generalization error) and the **solution** (regularization).

1. **The Problem: "Failure to adequately predict generalization error".** When you train a machine learning model, you give it a specific dataset to learn from (the training set). The "failure" mentioned in your sentence happens when a model gets a perfect score on the training data but fails miserably in the real world. Because the model thinks it's doing great based on its training score, it "fails to adequately predict" how poorly it will actually perform later.
2. **The Result: Overfitting.** This is when a model becomes a "memory champion" instead of a "problem solver." It learns the noise, the outliers, and the accidental patterns in the training data rather than the actual underlying logic.

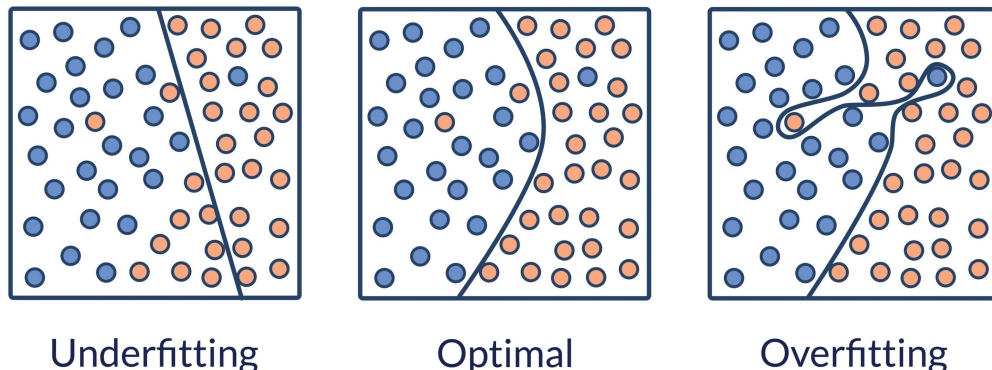


Figure 3.2: Underfitting vs Overfitting. (Image generated by Gemini)

3. **The Solution: Regularization.** Regularization is the "penalty" we introduce to keep the model humble. It discourages the model from becoming too complex or relying too heavily on any single piece of data. Mathematically, if we are trying to minimize a loss function $J(\theta)$, regularization adds a penalty term:

$$J(\theta) = \text{Loss}(\text{Data}, \text{Model}) + \lambda \Omega(w)$$

λ : Controls how much you want to penalize complexity.

$\Omega(w)$: The penalty term (like L1 or L2 regularization) that keeps the weights w small.

3.6.4 Modelling Structure

There are a whole variety of ways to model the structure in your data. DM often deals with methods that give good predictive accuracy at the expense of understanding the model (NN, SVM, boosted /bagged trees).

- Regression
- Neural networks (NN)
- Support Vector Machines (SVM)
- Generalized Additive Modelling
- Tree based methods
- Model averaged methods

3.7 Regression Models

We will often create models that explain how one random variable Y is affected by one or more other variables $x_1, x_2, \dots, x_p$. Y is the *response variable* and the x variables are called the *explanatory variables* (sometimes also called *predictor variables* or *covariates*). Models are used for either:

- **Description:** interested in better understanding the problem under study.

- **Prediction:** predict values of Y using the explanatory variable(s).

Why use regression models?

- They are easy to apply and interpret.
- The mathematical theory underlying them is well understood.
- They often provide an adequate approximation to reality.

3.7.1 Simple Linear Regression

We might want to model the relationship between the mean value of Y , and a single explanatory variable x using the simple linear regression model (3.1). A single neuron without activation basically performs linear regression, so simple linear regression can be viewed as the simplest neural network (one neuron).

$$E[Y_i] = \beta_0 + \beta_1 x_i + \epsilon_i \quad (3.1)$$

Where:	Symbol	Description
	Y_i	Dependent variable (target variable)
	x_i	Independent variable (input feature)
	β_0	Intercept of the regression model
	β_1	Slope coefficient
	ϵ_i	Random error term
	n	Number of observations

3.7.2 Parameter Estimation least squares

The method of **ordinary least squares (OLS)** involves drawing the line so that the sum of squared deviations of the original observations from the fitted line is minimized. How to choose β_0 and β_1 to minimize the value in (3.2).

$$\min \left(\sum_{i=1}^n (y_i - \hat{y}_i)^2 \right) \quad (3.2)$$

```
import pandas as pd
import statsmodels.formula.api as smf
data = pd.read_csv("data.csv")
model = smf.ols(formula='response ~ predictor', data=data).fit()
model.summary()
```

Listing 3.1: Sample OLM in statsmodels

3.7.3 Model Assumptions

if the following four assumptions about the errors are all true:

1. the expected value of the errors is a constant = 0 ($E[\epsilon_i] = 0$);
2. the errors ($\epsilon_1, \dots, \epsilon_n$) are independent;
3. the error variance, $\text{var}(\epsilon_i) = \sigma^2$, is an (unknown) constant;
4. the errors ($\epsilon_1, \dots, \epsilon_n$) are normally distributed;

then the least square estimate will be: **Unbiased** and **normally distributed**, so we can calculate confidence intervals and hypothesis tests for β_0 and β_1 .

Checking model assumptions

- We can check (1) by looking for trends in residuals vs fitted values (or vs predictor variable) as a useful diagnostic.
- We can check (2) by examining residuals vs time order (if known). Long sequences of positive or negative residuals suggests that the errors are dependent.
- We can check (3) by examining the spread of residuals. The spread of residuals should not vary systematically (heteroscedasticity).
- We can check (4) using a normal Q-Q plot. Points should stay close to a straight line.

t-test and p-value

In statistics, a p-value is a tool used to help decide whether the results you see in an experiment are likely due to a real effect or just random chance.

- A low p-value (typically ≤ 0.05): This means the results you observed are very unlikely to happen if the null hypothesis were true. Because the result is so surprising under the "nothing is happening" assumption, we usually "reject" that assumption and conclude that there is a statistically significant effect.
- A high p-value (typically > 0.05): This means your results are quite likely to occur just by random chance. In this case, you "fail to reject" the null hypothesis; you don't have enough evidence to claim a real effect.

It is very common to misunderstand p-values. Here are three things p-values do not tell you:

- **They don't measure the size of the effect:** A tiny, trivial difference can have a very low p-value if your sample size is massive.
- **They don't tell you the probability that your hypothesis is true:** A p-value is about the *data*, not the *hypothesis*.
- **The 0.05 threshold is arbitrary:** It is a standard convention in science, but it isn't a law of nature. A p-value of 0.051 is not fundamentally different from 0.049, yet it often determines whether a study gets published.

3.7.4 Robust Linear Regression

If regularization is about preventing a model from being too "extra" with its patterns, Robust Linear Regression is about preventing a model from being "bullied" by outliers. In standard Ordinary Least Squares (OLS) regression, the goal is to minimize the sum of the squares of the vertical distances (residuals) between the data points and the line. Because we square those distances, a single point that is very far away (an outlier) has a massive influence on the line, pulling it away from the majority of the data. Robust regression is designed to be "robust" (resistant) to these outliers. It doesn't let one or two "weird" data points ruin the trend for everyone else.

Table 3.3: Core Differences of OLS and RLM¹

Feature	Ordinary Least Squares (OLS)	Robust Linear Regression
Goal	$\min \sum (residual)^2$	Minimize a function that "down-weights" large residuals
Sensitivity	Highly sensitive to outliers	Resistant to outliers
Error Assumption	Assumes normal distribution	Works well with heavy-tailed distributions ²

¹ Robust Linear Model

² Heavy-tailed distributions are probability distributions with tails that decay slower than an exponential distribution.

Instead of giving every data point equal importance, robust regression uses an iterative process (often called **Iteratively Reweighted Least Squares**). The steps are:

1. Fit a line.
2. Look at the residuals (how far each point is from the line).
3. **Give less "weight"** to the points with huge residuals.
4. Re-fit the line using these new weights.
5. Repeat until the line stays put.

There are several mathematical ways to achieve this "robustness":

- **M-Estimation (Huber Loss):** This is the most common. It acts like OLS for small errors but acts like Mean Absolute Error (L1) for large errors. This means it doesn't "square" the big mistakes, so they don't pull the line as hard.
- **Theil-Sen Estimator:** It calculates the slopes between all pairs of points and chooses the median slope. Since it uses the median, it's incredibly hard for a few outliers to mess it up.
- **RANSAC (Random Sample Consensus):** It picks a random subset of data, fits a line, and sees how many other points "agree" with that line. It keeps the line that has the most "inliers."

Why use it?

Think of OLS as a group of friends trying to decide where to eat based on the average budget. If one friend is a billionaire, the "average" will be a 5-star restaurant nobody else can afford. Robust Regression is like the group choosing based on the median budget; the billionaire's outlier status is acknowledged, but it doesn't force the rest of the group into a situation that doesn't fit their reality.

3.7.5 Comparing Models

Models can be compared through:

- **R^2 :** coefficient of determination. It represents the percentage of the response variability explained by the model (higher is better).
- **AIC:** Akaike Information Criterion (smaller is better).
- **BIC:** Bayesian Information Criterion (smaller is better).

Table 3.4: Summary of R^2 , AIC and BIC

Metric	Goal	Better?	Best for ...
R^2	Goodness of fit	Higher	Seeing how well you fit the current data.
Adjusted R^2	Fit vs. Complexity	Higher	Comparing models with different numbers of predictors.
AIC	Prediction Quality	Lower	Picking a model that generalizes well.
BIC	Simplicity	Lower	Picking the simplest "true" model.

3.7.6 Multiple Linear Regression

Multiple Linear Regression is a statistical method used to model the relationship between one dependent variable and multiple independent variables.

3.8 Missing Values in Regression

Missing data may cause problems with your models. Data can be:

- Missing Completely at Random (MCAR),
- Missing at Random (MAR), or

- Missing Not at Random (MNAR)

and each type requires different handling approaches.

- Deletion methods like **listwise deletion (row removal)** are straightforward, but they can lead to significant loss of information and potential bias.
- **Imputation** techniques offer alternatives that preserve sample size but come with their own computational and statistical tradeoffs.

Table 3.5: MCAR vs MAR

Feature	MCAR	MAR
Depends on observed data?	No	Yes
Depends on missing values?	No	No
Bias if ignored?	No	Yes
Difficulty	Easy	Moderate

3.8.1 Imputation Methods

- **Mean/Median Imputation.** Replace missing values with central tendency. (we may also think of this as 'single imputation')
Advantages: Simple, works reasonably well with MCAR missing data.
Disadvantages: Underestimates variance (artificial reduction in standard errors), distorts relationships (bias correlations towards zero).
- **Single model-based Imputation** (e.g. regression imputation, k-NN). Predict missing values using other variables.
Advantages: More accurate than simple mean/median imputation, preserves variable relationships better than mean imputation
Disadvantages: Can overfit if model is too complex, requires sufficient data to build reliable models.
- **Multiple Imputation.** Create multiple datasets with different imputations.
Advantages: Accounts for uncertainty in missing values, preserves relationships between variables, gives realistic estimates of standard errors.
Disadvantages: More complex to implement, computationally intensive, requires careful specification of the imputation model, may be overkill for simple missing data patterns.

3.9 Multicollinearity

Multicollinearity is when predictors variables are correlated (i.e. a change in one x-variable is associated with a shift in another). This causes the following problems:

- Parameter estimation becomes unstable (small changes in data may lead to very different parameter estimates).
- Reduced precision of the estimates (so p-values of hypothesis tests are unreliable).

3.9.1 Variance Inflation Factor (VIF)

VIF (3.3) measure is often used to determine the severity of multicollinearity. It is calculated for each predictor and uses the other predictors to see how well it can be modelled.

$$\mathbf{VIF}_j = \frac{1}{(1 - R_j^2)} \quad (3.3)$$

where the R_j^2 is the coefficient of determination for model: $x_1 = \beta_0 + \beta_2 x_2 + \beta_3 x_3$

If multicollinearity is a problem, consider reducing the number of variables by either:

Table 3.6: Some Common Rules of Thumb

Condition	Interpretation
$VIF < 1$	No correlation
$1 < VIF < 5$	Moderate correlation (generally acceptable)
$VIF > 5$	Highly correlated (potentially problematic)

- dropping highly correlated predictors (**Feature Selection**), or
- doing a **PCA**.

3.10 Curse of Dimensionality

The Curse of Dimensionality refers to problems that occur when the number of features (dimensions) in a dataset becomes very large. As dimensionality increases, data becomes sparse, distances become less meaningful, and many machine learning algorithms perform worse. The term was introduced by Richard Bellman when studying optimization problems. Dimensionality reduction has many advantages, such as:

- Fewer features potentially imply less complexity.
- Less storage space.
- It speeds up the analyses.
- Model accuracy improves due to less misleading data.
- It removes noise and redundant features.

So we need to reduce dimensions, there are at least two types of methods: **Feature Selection** and **Feature Extraction**.

scikit-learn vs statsmodels

The main difference between **scikit-learn (sklearn)** LinearRegression and **statsmodels** OLS (all python libraries) is their purpose:

- **scikit-learn:** machine learning / prediction / cross-validation / feature selection
- **statsmodels:** statistical inference / analysis / regression diagnostics / hypothesis testing

3.11 Feature Selection

I call the following three methods: **FEW** methods

- **Filter Methods.** Also known as **single factor analysis**. It uses a ranking or sorting algorithm to filter out those features that are less useful. The predictive power of each individual variable is evaluated:
 - Correlation Threshold. Pearsons Correlation measures linear correlation between two variables. (quantitative to quantitative)
 - ANOVA F-value (quantitative to categorical)
 - Chi Square test (categorical to categorical)
- **Wrapper Methods.** The attributes subset selection is done using the learning algorithm and generally selects features by directly evaluating their impact on the performance of a model. We have those methods:
 - Forward selection.
 - Backward selection (or backward elimination).
 - Stepwise selection (combination of forward and backward selection).

- **Embedded Methods.** Embedded methods use algorithms that have built-in feature selection methods. For instance, LASSO¹ (Section 3.13.1).

3.11.1 Pearson's Correlation

Pearson's Correlation Coefficient, often denoted by the letter r , is a statistical measure that quantifies the strength and direction of the linear relationship between two continuous variables. It is the most widely used correlation metric and serves as a cornerstone in both statistics and data science for feature selection.

The Value Scale

The value of r always ranges from -1 to 1. Here is how to interpret the results:

- **$r = 1$** (Perfect Positive Correlation): As variable X increases, variable Y increases in a perfectly linear fashion.
- **$r = -1$** (Perfect Negative Correlation): As variable X increases, variable Y decreases in a perfectly linear fashion.
- **$r = 0$** (No Linear Correlation): There is no linear trend between the variables.

The Mathematical Formula

Mathematically, it is defined as the covariance (Section 6.8) of the two variables divided by the **product of their standard deviations**. This "normalization" ensures that the result is dimensionless and unaffected by the scale of the data (e.g., measuring in meters vs. centimeters won't change r).

$$r = \frac{\text{cov}(X, Y)}{\sigma_X \sigma_Y} = \frac{\sum (X_i - \bar{X})(Y_i - \bar{Y})}{\sqrt{\sum (X_i - \bar{X})^2 \sum (Y_i - \bar{Y})^2}} \quad (3.4)$$

Key Assumptions (The "Catch")

For Pearson's correlation to be valid and meaningful, your data should meet these criteria:

- **Linearity:** The relationship between variables must be a straight line. If the relationship is curved (like a U-shape), Pearson will fail to capture it.
- **Continuity:** Both variables should be continuous (interval or ratio scale).
- **No Outliers:** Pearson is highly sensitive to outliers. A single extreme data point can significantly inflate or deflate the correlation coefficient.
- **Normality:** Ideally, both variables should follow a normal distribution.

Interpretation of Correlation Coefficient

Table 3.7: interpretation for the absolute value of r

Absolute Value of Correlation	Relationship
0.000 - 0.199	Very weak
0.200 - 0.399	weak
0.400 - 0.599	Moderate
0.600 - 0.799	Strong
0.800 - 1.000	Very String

Conclusion: if two variables are correlated, one of them can be removed from further analyses.

¹LASSO is a selection method, while RR is not, but they both belongs to shrinkage methods in terms of mathematics.

3.12 Feature Extraction

Feature selection drops some redundant variables. Might lose important information that was uniquely reflected in the eliminated variables. Feature extraction means creating new features by transforming the original ones.

- Original features are combined or transformed.
- New features represent the same information in fewer dimensions.

3.12.1 PCA (Principal Component Analysis)

To understand how Principal Component Analysis (PCA) works in a machine learning pipeline, it is important to distinguish between the training phase and the prediction (testing) phase. You must apply the same transformation to all datasets, *but the transformation parameters are derived solely from your training set*.

Because PCA finds an axis that maximizes the variance, it is typically important that all the variables are normalized or standardized. This ensures each predictor variable has the same influence in the PCA.

The axis is chosen to maximize the variability of points when they are projected onto it. This is the maximum information of the data cloud that can be found in a single axis. A second axis can be chosen that is orthogonal (at right angles) to the 1st. The 2nd axis is also chosen to maximize variation. The positions of the data on the PCA axes are known as the PCA scores. These may be used as an input variables. This has important advantages:

- Each axis is orthogonal to the others.
- This means they are all uncorrelated.
- This removes problems with multi-collinearity.

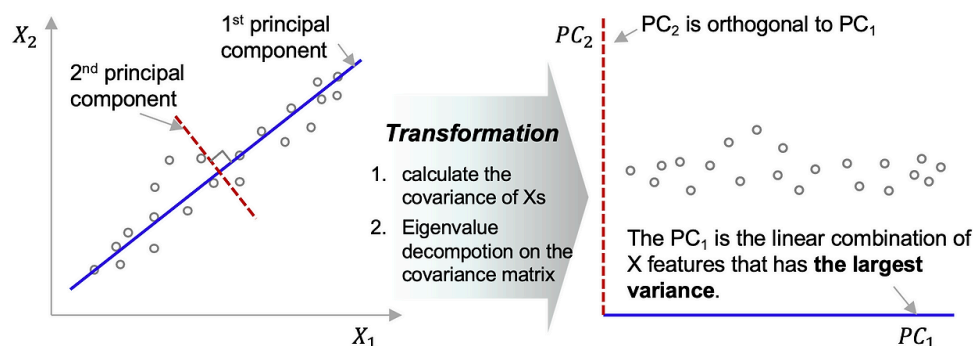


Figure 3.3: from: <https://rfriend.tistory.com/751>

Principles to do PCA:

- Do PCA on the *covariance matrix* (Table 6.4) if variables are dimensionally homogeneous and on similar scales.
- Do PCA on the *correlation matrix* if variables are not dimensionally homogeneous or are on different scales.

3.12.2 Biplot

Biplot vectors can be used to give context to the PCA by show how original variables influence the PCs:

- Longer vectors indicate a stronger contribution, the angle between vectors shows correlations (smaller angles mean higher correlation).
- Vectors pointing in the same direction as a data cluster suggest that those variables are important in defining that cluster.

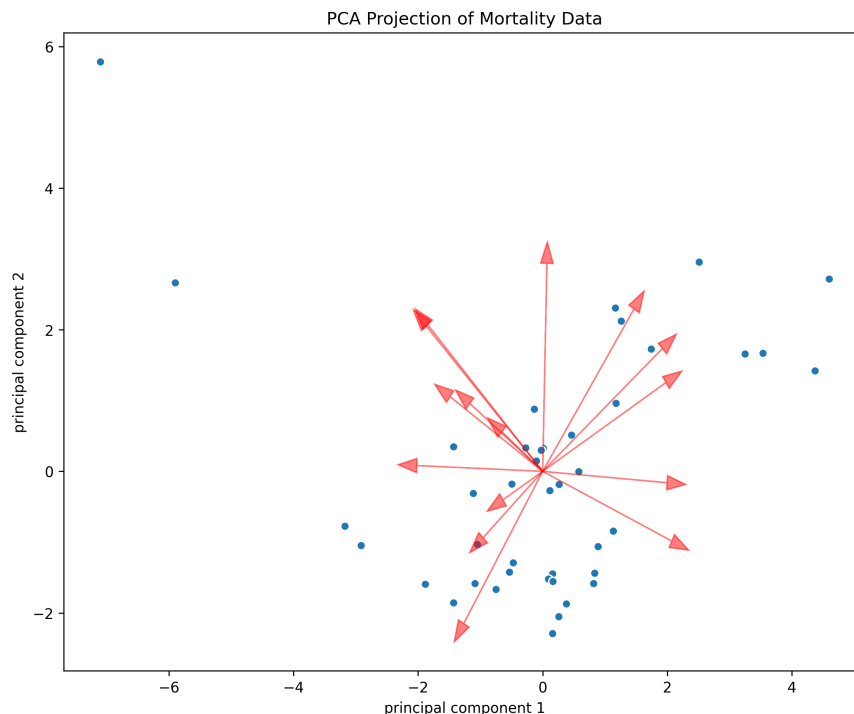


Figure 3.4: Data displayed in first two principal components, and its relations with previous features.

3.13 Non-Numeric Input Coding

To represent non-numeric inputs in a model, you must convert them to some sort of numeric values. This conversion is most commonly done by creating design variables (or dummy variables), with each design variable representing approximately one level of the categorical input. **One-hot encoding** (Table 3.8) is one of the most popular methods (others exist). For each unique category, create a new binary column (0s and 1s). Each row gets a 1 in the column corresponding to its category and 0s elsewhere.

Table 3.8: Dataset before and after One-Hot Encoding

Original Dataset		One-Hot Encoded Dataset		
	Color	Red	Green	Blue
0	Red	1	0	0
1	Blue	0	0	1
2	Green	0	1	0
3	Blue	0	0	1
4	Red	1	0	0

3.13.1 Shrinkage methods

Shrinkage methods are a subset of **Regularization** techniques in statistics and machine learning. The term "shrinkage" refers to the act of adding a penalty to the model's loss function, which "shrinks" the estimated coefficients toward zero (or exactly to zero). This is a powerful tool for balancing the bias-variance tradeoff, particularly when you have many features (high dimensionality) or when your features are highly correlated (multicollinearity). There are basically two primary methods: **Ridge Regression** (L_2 regularization) and **LASSO Regression** (L_1 regularization). Both methods modify the *loss function* by adding a penalty term: $\text{Loss} = \text{RSS} + \text{Penalty}$.

- **Ridge Regression (L_2 Regularization or RR)²:** This adds a penalty equal to the sum of the squares of the coefficients $\lambda \sum (\beta_j^2)$. It shrinks all coefficients toward zero, but they never actually reach zero. Best for when you have many features that all contribute slightly to the output, or when features are highly correlated. It keeps all variables in the model but limits their influence.

$$\sum_{i=1}^n (y_i - \hat{y})^2 = \sum_{i=1}^n (y - [\beta_0 + \sum_{j=1}^p (\beta_j x_{ij})])^2 + \lambda \sum_{j=1}^p \beta_j^2 = \mathbf{RSS} + \lambda \sum_{j=1}^p \beta_j^2$$

- **LASSO Regression (L_1 Regularization)³:** This adds a penalty equal to the sum of the absolute values of the coefficients $\lambda \sum |\beta_j|$

$$\sum_{i=1}^n (y_i - \hat{y})^2 = \sum_{i=1}^n (y - [\beta_0 + \sum_{j=1}^p (\beta_j x_{ij})])^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- **The Tuning Parameter (λ):** In both methods, the strength of the shrinkage is controlled by a hyperparameter, usually denoted as λ
 - **If $\lambda = 0$:** The penalty has no effect, and you get a standard OLS regression.
 - **If $\lambda \rightarrow \infty$:** The penalty dominates, and all coefficients shrink toward zero (making the model very simple).
 - **Selection:** We typically find the optimal λ using *Cross-Validation*.

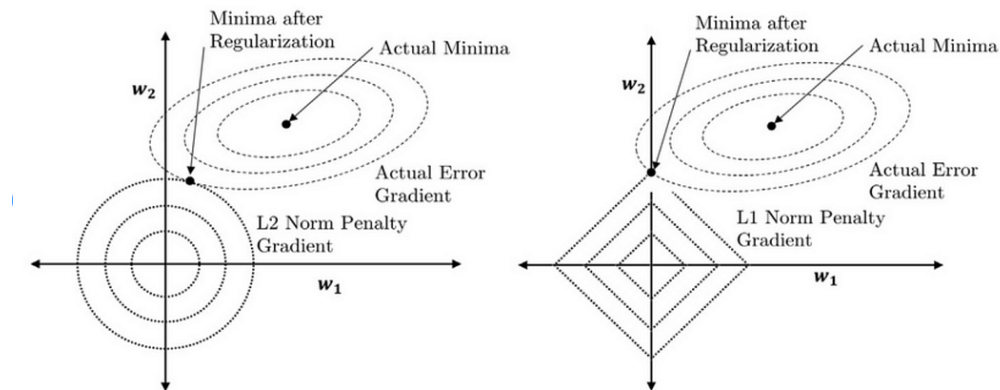


Figure 3.5: Illustration of why LASSO can set coefficients to zero.

Why do we need to "shrink" coefficients?

In a standard Ordinary Least Squares (OLS) regression, the model tries to minimize the Residual Sum of Squares (RSS). If you have many predictors, OLS can become very sensitive to noise in the training data, leading to:

- **Overfitting:** The model performs perfectly on training data but fails on new data.
- **High Variance:** Small changes in the training data lead to huge changes in the estimated coefficients.

Shrinkage introduces a small amount of bias into the model to significantly reduce its variance, resulting in better overall prediction accuracy.

²<https://www.youtube.com/watch?v=Q81RR3yKn30>

³<https://www.youtube.com/watch?v=NGf0voTmlcs>

Table 3.9: Comparison: PCA vs. Shrinkage Methods

Criteria	PCA	Shrinkage (Lasso/Ridge)
Approach	Coordinate Transformation	Coefficient Regularization
Variables	New synthetic components	Original features preserved
Supervision	Unsupervised (ignores y)	Supervised (uses y)
Interpretability	Low	High

Note: While both reduce overfitting, PCA is a data-driven reduction while Shrinkage is a task-driven constraint.

3.14 Generalized Linear Models (GLMs)

GLMs are a powerful statistical framework that extends ordinary linear regression. While standard linear regression assumes your data follows a normal (Gaussian) distribution, real-world data often does not. GLMs allow us to model these different types of data by relaxing those strict assumptions.

GLM hold a "cornerstone" position in both *classical statistics* and *modern machine learning*. They act as the essential bridge between simple linear regression and complex neural networks. If you look at a single neuron in a Deep Learning model, you are essentially looking at a GLM:

$$\textbf{Linear Combination} : z = w^T x + b$$

$$\textbf{Activation Function} : a = \sigma(z)$$

For instance, a neural network using a Sigmoid activation for a binary output is mathematically identical to a Logistic Regression GLM. Understanding GLMs provides the foundational logic for why we use specific activation functions for specific output layers (e.g., Softmax for multi-class, Exponential for counts).

3.14.1 The Three Core Components

Every GLM is built using three specific "bricks":

1) The Random Component

This defines the probability distribution of your target variable (y). In a GLM, y must come from the Exponential Family, which includes:

- **Normal:** For continuous data (height, temperature).
- **Bernoulli/Binomial:** For binary data (click vs. no-click).
- **Poisson:** For count data (number of emails received per day).

2) The Systematic Component

A familiar linear combination of your input features (3.5). It represents the "linear predictor" that sums up the influence of your variables.

$$\eta = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k \quad (3.5)$$

3) The Link Function

This is the "bridge" that connects the linear predictor (η) to the expected value of your data (η). It ensures that the linear output makes sense for the distribution you're using.

- **Logit Link:** Squashes the linear output into a range between 0 and 1 (used for Logistic Regression).

$$\text{logit}(p) = \ln \left(\frac{p}{1-p} \right) = \beta_0 + \beta_1 x_1 + \cdots + \beta_k x_k$$

- **Log Link:** Ensures the output is always positive (used for Poisson Regression).

$$\ln(\mu_i) = \beta_0 + \beta_1 x_1 + \dots + \beta_k x_k$$

3.14.2 Overdispersion

Overdispersion is a statistical phenomenon that occurs when the actual variability (variance) in your data is higher than what your chosen probability model predicts. If you are modeling the number of emails received per hour, and your mean is 10, the Poisson model (Section 6.5.3) expects the variance to also be 10. If the actual variance in your data is 25, you have over-dispersion. If you ignore over-dispersion, your GLM will become "over-confident", because:

- **Standard Errors are too small:** The model thinks it is more precise than it actually is.
- **P-values are artificially low:** You might conclude a variable is "statistically significant" when it is actually just random noise.
- **Narrow Confidence Intervals:** Your "safety margins" for predictions will be too thin, leading to failures in real-world applications.

The most common way to detect over/under-dispersion after fitting a model is to look at the **Dispersion Parameter** (ϕ) (6.7). if $\phi > 1$ means overdispersion, otherwise underdispersion.

3.14.3 Data Preprocessing

In logistic regression, the model attempts to find a decision boundary that best separates your classes by maximizing the likelihood of the observed data. When one class significantly outnumbers the other (e.g., 99% "No Disease" vs. 1% "Disease"), the "math" behind the model can become biased toward the majority. Performance can be improved by either scaling down the majority (genuine) class or creating new data for the minority (fraud) class. Here are several methods:

Doing Nothing

- + Maintains the original data distribution, avoiding artificial modifications.
- + If the dataset is large enough, some models (e.g., tree-based models) can handle imbalance reasonably well.
- + No risk of losing information (as in undersampling) or introducing synthetic noise (as in oversampling).
- Many models (e.g., standard classifiers like logistic regression, SVMs, or neural networks) will bias predictions towards the majority class, leading to poor recall for the minority class.
- Metrics like accuracy may be misleading. A classifier predicting only the majority class could still appear to perform well.

Undersampling

- + Speeds up training - reduced dataset size.
- + Can improve model performance if the majority class contains redundant or noisy samples that don't contribute much useful information.
- + Helps with models that struggle with large datasets due to memory constraints.
- Risk of losing valuable information by discarding majority-class instances.
- If the dataset is already small, this can make the model underfit because it has too little data to learn from.
- Works best when the majority class has a lot of redundant examples, so removing some doesn't hurt model performance

Oversampling

Oversampling (Increasing the Minority Class, e.g., RandomOverasmping, SMOTE), note on oversampling order:

In supervised learning (classification), oversampling must be applied after splitting to prevent data leakage. Here we are doing unsupervised clustering on the whole dataset, so applying oversampling to all data before clustering is appropriate.

- + Retains all original data giving the model more minority class examples to learn from.
- + Can significantly improve the model's ability to detect minority class cases.
- + SMOTE (Synthetic Minority Over-sampling Technique) and other advanced methods create synthetic instances, reducing the risk of overfitting compared to simple duplication.
- Increases training time due to the larger dataset.
- Can introduce synthetic noise if new instances aren't well-representative of real data.
- If oversampling is done before splitting the dataset, it can lead to data leakage, where synthetic samples make test performance look better than it really is.

Hints

- If your dataset is small and overfitting is a concern → *Use SMOTE*.
- If SMOTE creates unrealistic samples → *Try random oversampling*.
- For categorical-heavy datasets → *Random oversampling might be easier*.

3.15 Unsupervised and Supervised

Machine learning techniques are usually divided in two categories:

1. **Unsupervised:** no response variable is available to the learning algorithm. Clustering and PCA are examples of unsupervised learning.
2. **Supervised:** response variable(s) are given to the learning algorithm. Regression / Naïve bayes / LDA / Decision Trees / Neural Networks are examples of supervised learning.

3.16 Clustering Analysis

Clustering is the classification of objects into different groups, or more precisely, the partitioning of a data set into subsets (clusters), so that the data in each subset (ideally) share some common trait - often according to some defined distance measure. Clustering is the process of grouping a set of objects into classes of similar objects.

The idea of clustering analysis is to find groups of objects such that the objects in a group are similar (or related) to one another and different from (unrelated to) objects in other groups.

Can be thought of as treating all variables as predictor variables and using these to invent a new categorical response variable, i.e. a classification. To do this:

- intra-cluster (intra-class) distances are minimized. **Intra** comes from the Latin word for "within." In clustering, this refers to the distance between data points that belong to the same group.
- inter-cluster (inter-class) distances are maximized. **Inter** comes from the Latin word for "between" (like an interstate highway that goes between states). This refers to the distance between different clusters.

Part of a broader group of techniques, often called **segmentation**. However, in reality, the notion of a cluster can be ambiguous. How many clusters would you consider appropriate?

3.16.1 Distance Measures

- Results from a function applied to the variables of each pair of objects. This is what the clustering algorithm acts upon to assign data into groups.
- There are many different measures of distance, or dissimilarity.

- The distance quantifies the difference between each pair of objects.
- The choice of distance measure will (most likely) affect the clustering outcome.

different distances

- Euclidean distance
- Manhattan distance
- Cosine similarity
- Mahalanobis distance

3.16.2 Methods of Cluster Analysis

Table 3.10: The Four Primary Categories of Clustering Methods

Category	Key Approach	Primary Example
Partitioning	Minimizes distance to a central point	K-Means, Fuzzy C-Means
Hierarchical	Builds a nested tree (Dendrogram)	Agglomerative, Divisive
Density-based	Identifies high-density regions	DBSCAN
Distribution-based	Uses statistical distributions	GMM ¹

¹ Gaussian Mixture Models provide "soft" clustering assignments based on probability.

Partitioning Methods

Table 3.11: K-Means Clustering Pros and Cons

Pros	Cons
• Very fast and intuitive • Easy to implement • Often comes up with good, if not best, solutions • Can assign new cases to clusters	• Applicable only when mean is defined, what about categorical data? • Need to specify k in advance (but this can be tuned) • Sensitive to outliers • Only good for clusters with convex shapes and similar sizes • Sensitive to initial starting points, may not find optimal solution

How to determine k, so that we can operate K-Means method?

Elbow method Calculate within-Cluster Sum of Square (**WCSS**) for a series of k values. **WCSS** is the sum of squared distances between each point and the centroid in a cluster. As the number of clusters increases, the **WCSS** value decreases. The optimal value is in the so called elbow point, where for higher k, the **WCSS** does not decrease much.

Silhouette score The silhouette value measures the quality of a clustering solution, for example it determines how well an object lies within its own cluster (cohesion) compared to other clusters (separation). The silhouette ranges from -1 , $+1$.

- A high value indicates that the object is well matched to its own cluster and poorly matched to neighbouring clusters.
- If most objects have high values, then the clustering configuration is appropriate.
- If many points have low or negative values, then the clustering configuration may have too many or too few clusters.

- The silhouette for observation i is:

$$s(i) = \frac{b(i) - a(i)}{\max[a(i), b(i)]} \quad (3.6)$$

Where:

$a(i)$ = intra-cluster distances

$b(i)$ = inter-cluster distances

Clustering methods: k-Means, k-Medoids, Model based methods

Hierarchical Methods

It produces a set of nested clusters organized as a hierarchical tree. It can be visualized as a dendrogram, a tree-like diagram that records the sequences of merges or splits. Two main types of hierarchical clustering:

- Agglomerative (bottom-up):
 - Start with the points as individual clusters
 - At each step, merge the closest pair of clusters until only one cluster (or k clusters) left
- Divisive (top-down):
 - Start with one, all-inclusive cluster
 - At each step, split a cluster until each cluster contains a point (or there are k clusters)

Agglomerative Clustering Agglomerative clustering is a bottom-up hierarchical clustering strategy. Instead of starting with a fixed number of groups, it begins by treating every single data point as its own individual cluster and then systematically merges them until a stopping criterion is met. More popular hierarchical clustering technique (simpler mathematically) and widely considered to work better than divisive clustering. Basic algorithm is straightforward:

- Compute the distance matrix
- Let each data point be a cluster
- Repeat
- Merge the two closest clusters
- Update the proximity matrix
- Until only a single cluster remains

Key operation is the computation of the proximity of two clusters. Different approaches to defining the distance between clusters distinguish the different algorithms

Because we treat every single data as a cluster, then how do we update the proximity matrix? We use at least four methods:

- **MIN (single-link)** calculate minimum distance between two clusters;
- **MAX (complete-link)** calculate maximum distance between two clusters;
- **Group Average (average-link)** calculate distance of every pairs, and average them;
- **Distance Between Centroids (centroid)** –

Summary

- **Strengths**
 - Dendrogram highlights relationships between groups.
 - Don't have to choose k a priori. Can cut the tree at any level.
- **Weaknesses**
 - Can be infeasible for very large datasets. Generally $\mathcal{O}(n^3)$ or $\mathcal{O}(n^2 \log n)$.
 - Results dependent on particular algorithm used.

- Not easy to assign new cases.⁴

Density-Based Clustering Methods

Clustering based on density (local cluster criterion), such as density-connected points or based on an explicitly constructed density function

Major features:

- Discover clusters of arbitrary shape
- Handle noise
- One scan
- Need density parameters
- Several interesting studies:
 - DBSCAN: Ester, et al. (KDD96)
 - DENCLUE: Hinneburg & D. Keim (KDD98/2006)
 - OPTICS: Ankerst, et al (SIGMOD99).
 - CLIQUE: Agrawal, et al. (SIGMOD98)

3.16.3 Summary

- Cluster analysis forces data into groups. This does not necessarily mean that the groups exist.
- It is done via a distance measure, of which many exist.
- Standardisation is often, though not always, advisable when variables have different ranges.
- There are two main approaches: hierarchical and partitioning.
- It can be considered subjective. Must make (fairly arbitrary) choice of distance measure, clustering algorithm, k (if k-means), or group joining criterion (if hierarchical).

3.17 Decision Tree

3.17.1 Supervised and Unsupervised Learning

Supervised learning and unsupervised learning are two major categories of machine learning, **based on whether the training data has labels**.

Supervised Learning

In supervised learning, the model learns from input data + correct answers (labels). The algorithm learns the relationship between input and output, then predicts labels for new data. The main Tasks are: Classification.

Table 3.12: Main Algorithms of Supervised Learning

Classification	Regression
Logistic Regression	Linear Regression
Decision Tree	Ridge / Lasso
Random Forest	Random Forest Regressor
Support Vector Machine (SVM)	Neural Networks
Neural Networks	

⁴A potential solution could be the use of a supervised machine learning technique, using the output from the agglomerative algorithms as labels, and then classify the new observations accordingly.

2. Unsupervised Learning

In unsupervised learning, the data has no labels. The model tries to discover hidden patterns or structure. Main Tasks is Clustering. Group similar data points together.

Algorithms:

- K-Means
- Hierarchical Clustering
- DBSCAN

3. Key Difference

Table 3.13: Supervised VS Unsupervised

Feature	Supervised	Unsupervised
Labels available	Yes	No
Goal	Predict outputs	Find hidden patterns
Example	Price prediction	Customer groups
Evaluation	Accuracy, RMSE	Silhouette score, visual checks

3.18 Naive Bays

Thomas Bays introduced Bays Theorem in 1700s. Each data feature contributes to a portfolio of evidence. Assumes that all data features are statistically independent of each other. Probability of event H given evidence E:

$$Pr[H|E] = \frac{Pr[E|H]Pr[H]}{Pr[E]} \quad (3.7)$$

Where:

P(E) = Evidence

P(H|E) = Posterior probability (probability of hypothesis H given evidence E)

P(H) = Prior probability of hypothesis H

P(E|H) = Likelihood (if hypothesis is true, what probablity will evidence occur.)

The conditional probability of an event A (given B) is the probability that an event A will occur given that another event, B, has occurred.

$$P(A|B) = \frac{P(A \cap B)}{P(B)} \quad (3.8)$$

3.18.1 Properties of Conditional Probability

3.18.2 Naive Bayes

- Models developed with Naive Bayes have good explanatory power, just as with Decision Trees
- In the credit scoring problem the Income and Marital Status attributes essentially determines the risk factor for a given individual (Why?)
- It is also true that the typical profile of a good risk applicant is someone who is married and has high income

3.18.3 Bayes Classifiers

$$c_{MAP} = \operatorname{argmax} P(c_j|x_1, x_2, \dots, x_n) \quad (3.9)$$

$$= \operatorname{argmax} \frac{P(x_1, x_2, \dots, x_n)P(c_j)}{P(x_1, x_2, \dots, x_n)} \quad (3.10)$$

$$= \operatorname{argmax} P(x_1, x_2, \dots, x_n|c_j)P(c_j) \quad (3.11)$$

MAP:maximum A posterior

3.18.4 General Characteristics of Naïve Bayes

- One of the most efficient classification techniques as it makes only one pass through training data
- Naïve Bayes works well in practise even though features may not be statistically independent
- In cases where a large number of features are dependent on each other, accuracy could drop substantially and a more advanced version called Bayesian Network should be used

3.19 Practical Issues in Classification

Several challenges that occur during the development of classification models:

- **Underfitting:** Occurs when a model is too simple (e.g., a decision tree with insufficient depth) to learn the underlying patterns in the training data, leading to poor test accuracy.
- **Overfitting:** Occurs when a model is overly complex and learns the training data "too well", including its noise. This results in high training accuracy but a drastic drop in performance on new data.
Causes: Noise in the training set or a lack of sufficient data to capture general patterns.
- **Missing Values and Costs:** Other practical considerations include how to handle missing data and the differing costs associated with different types of misclassification.

3.20 Methods for Performance Evaluation

To obtain reliable estimates of how a model will perform on unseen data, several estimation methods are used:

- **Holdout:** Data is split, typically 2/3 for training and 1/3 for testing.
- **Cross-Validation:** Data is partitioned into k subsets. The model is trained on k-1 partitions and tested on the remaining one (e.g., k-fold or leave-one-out).
- **Bootstrap:** A method involving sampling with replacement.
- **Occams Razor:** This principle suggests that given two models with similar generalization errors, the simpler model is preferred.

3.21 Methods for Model Comparison

When comparing two models (e.g., M_1 and M_2), researchers must determine if the difference in performance is statistically significant or merely due to random fluctuations:

- **Tests of Significance:** Performance depends on factors like class distribution and the size of training/test sets.
- **Error Rate Comparison:** By assuming independent test sets and a normal distribution for error rates, one can calculate a confidence interval for the true difference in performance:

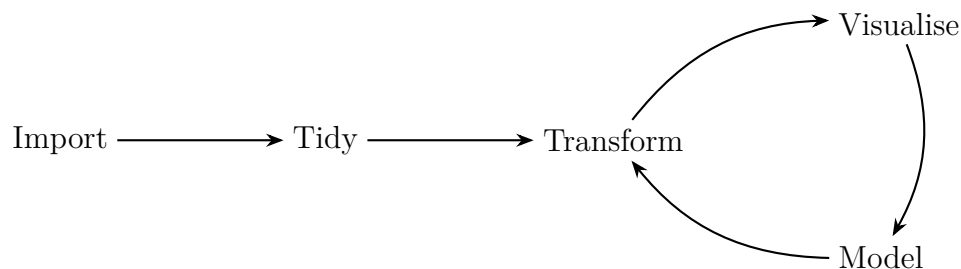
$$\sigma_t^2 = \frac{e_1(1 - e_1)}{n_1} + \frac{e_2(1 - e_2)}{n_2}$$

Chapter 4

Statistical Programming for Data Sciences(COMP828)

4.1 Descriptive Statistics

- Statistics is the art of learning from data. (Probably half art, half science)
- It is concerned with the collection of data, its subsequent description and analysis, which often leads to the drawing of conclusions.
- There are two main branches of statistics:
 - **Descriptive Statistics:** describing and summarizing of data
 - **Inferential Statistics:** drawing of conclusions from data
- In the first few weeks, we will mainly discuss descriptive statistics.
- It is often useful to summarize a dataset using a single number that represents all the observations.
- There are many ways to do this but the three most common are **mean**, **median** and **mode**¹, which are the measures of central tendency.
- The other group of descriptive statistics, which are frequently used, are **standard deviation**, **variance**, **range**, **quartiles**, and **interquartile range**.
- Other useful statistics are proportion, **minimum**, **maximum**, etc.



4.2 Plots

- **Barplots** can only be done on qualitative variables. A barplot is a tool to visualize the distribution of a qualitative variable.
- **Histogram** is the better option to see the distribution.

¹Mode is the most commonly occurring value in a distribution. Interestingly, there is no R function that can find the mode.

- **Boxplot** summarizes the data and provides some meaningful statistics.
- **Scatter** plot is useful to visualize the relationship of two quantitative variables.
- **Line** plot is useful to visualize and summarize the key characteristics of the time series data.

```
barplot(table(mtcars$gear))
hist(mtcars$gear)
boxplot(mtcars$wt)
plot(mtcars$mpg, mtcars$wt)
plot(Nile)
```

```
mtcars[which(mtcars$am==0),] # watch ',','
```

4.3 mode function in R

```
mode_stat <- function(x) {
  ux <- unique(x)
  ux[which.max(tabulate(match(x, ux)))]
}
```

4.4 Factors

it is extremely common and often necessary to explicitly transform numerical data into categorical types (called factors in R). In data science and machine learning, a number isn't always a "measurement"; sometimes it is just a label. If you leave these as numeric types, your statistical models will treat them mathematically (e.g., calculating an average), which can lead to incorrect results.

4.4.1 When You MUST Transform

You should transform a number to a categorical type in these three main scenarios:

- **Identifiers/Codes:** If your data uses 1 for "Male" and 2 for "Female," R will think 2 is "twice as much" as 1. You must tell R these are labels.
- **Likert Scales / Rankings:** If you have survey data (e.g., ratings 1-5), you may want to treat them as ordered categories rather than continuous numbers.
- **Low-Cardinality Integers:** If a column only contains values like 0 and 1 (like a "Success/Failure" outcome), converting it to a factor ensures models like Logistic Regression treat it as a classification target.

4.4.2 How to Do It in R

The standard way to perform this transformation is using `as.factor()`.

Basic Conversion

```
# Convert a numeric column to a factor
df$outcome <- as.factor(df$outcome)
```

Conversion with Descriptive Labels

It is often better to provide labels so your plots and summaries are easier to read.

```
df$outcome <- factor(  
  df$outcome,  
  levels = c(0, 1),  
  labels = c("Healthy", "At_Risk")  
)
```

4.4.3 Why It Matters for Models and Plots

Explicitly setting types changes how R functions behave:

- In Plotting (ggplot2): If a variable is numeric, R creates a color gradient (light blue to dark blue). If it is a factor, R assigns distinct colors to each group.
- In Modeling (lm or glm): R automatically creates "dummy variables" for factors. If you leave a category as a number, the model will try to find a linear slope between the categories, which is usually a logical error.
- In Summaries: `summary(df$numeric_col)` gives you Mean and Median. `summary(df$factor_col)` gives you a Frequency Count (like a table).

4.4.4 The "Binning" Technique

Sometimes you transform a continuous number (like Age) into categories (like "Young", "Middle-Aged", "Senior"). This is called discretization.

```
# Creating 3 categories from a continuous variable  
df$age_group <- cut(  
  df$age,  
  breaks = c(0, 18, 60, Inf),  
  labels = c("Minor", "Adult", "Senior")  
)
```


Chapter 5

Video and Image Processing(COMP820)

5.1 Assignment

5.1.1 Assignment 1 and 2 topic context suggestions: Privacy-preserving human activity recognition systems

The objective of this research task is the application of technology to advance society and quality of living. Ideally, the Assignment 2 topic is intended to be upgraded into a research publication. Therefore, you will be investigating the feasibility of an initial proof-of-concept associated with privacy-preserving infrastructures that could provide interpretable diagnostic feedback on observed human movement, which are applicable in the areas of rehabilitation, sport, ageing and other similar contexts.

The assignment should consider three aspects: human activity, environment and technology. For example, you could be developing and implementing HCI prototyping framework for integration of game controllers with open source software for the purpose of recognising body movements from the volume area in front of the TV/screen, stationary bicycle or treadmill and emulating joystick/game controller commands in real time.

Examples of Assignment research question:

- Can we develop user-configured video anonymization (e.g. pixelate faces)?
- Can we develop a low-cost privacy-preserving monitoring and alerting systems for elderly people living alone?
- Can technology improve end-user experience in order to promote physical activity?
- Can technology improve fall-prevention and fall-awareness with ageing population?
- Can technology improve safety for working environments, help to reduce incidents or augment safety inspections?
- Can we use pose estimation and existing video cameras to create a Kinect-equivalent universal game controller emulation?
- **Can a stationary bike (or any other prop e.g. tennis racquet, golf club) movements be extracted with a computer vision system (emulating joystick/game controller activity) and be converted into indoor/outdoor activity or be used to play an old video game?**
- Can we repurpose a low-cost hardware, use a familiar game as a low-cost alternative to monthly subscription to e.g. a cycling game?
- Can we advance the state-of-the-art of exergaming, enhance environments and user experience in various activities?

- Can we apply computer vision and human activity recognition (HAR) to improve or augment motor learning, daily or task-specific activities?
- How can we improve health and wellness in either self-isolation or outdoor activities?
- How can we apply computer vision, human activity recognition (HAR) and pose recognition to promote active life in self-isolation, or assisting a growing number of people living on their own?
- How can we apply computer vision, human activity recognition (HAR) or pose recognition to monitor and extract privacy-preserving time-series data that carry real-time diagnostic information on selected activities for people living on their own?

5.1.2 How to do it

Normally, you write a full thesis or paper and then make a poster summarizing it. But in assignment 1, you are essentially making a mini-thesis in condensed poster form, so the poster comes first as an individual literature review and critical analysis. Think of it as a preview of a bigger project. Here's how it works step by step:

1) The Poster is Your Mini-Thesis

Even though it's called a poster, it's really a research synthesis in visual format. It doesn't have to have results from experiments you are summarizing and analyzing other people's research. Think of it as telling the story of the research area in concise, visual form. It should include:

- What the topic is and why it's important
- Key methods or technologies used in the field
- Strengths, weaknesses, and challenges
- Future directions or gaps in research
- References (APA style)

2) How to Make a Poster Before Doing a Full Thesis

You don't need original results yet, but you need:

1. **A clear topic / research question**
2. **A literature review** at least 5-10 academic sources
3. **Critical analysis** compare, contrast, and discuss trends
4. **A visual layout** figures, diagrams, tables to explain ideas
5. **Coherence and justification** explain why this research area is important

3) Benefits of Doing It This Way

- Forces you to focus on key concepts and trends before you do experiments.
- Gives you a clear roadmap for the larger project (Assignment 2 or future thesis).
- Helps you identify gaps in research that your project could fill.
- Prepares you to communicate complex ideas clearly to peers or supervisors.

Analogy

Think of it like an architectural model before building the house: You research and design first. Then show a model (poster) with your ideas, comparisons, and plan. Later, you can build the full house (Assignment 2 or thesis) with experiments or new contributions.

5.2 Self-directed Learning: Matlab Resources for Image Enhancements

- Deblurring Images Using a Wiener Filter

- Noise Removal Links to an external site.
- Basic Image Enhancement and Analysis Techniques

5.3 Homework and self-paced learning courseware

Learn Matlab at your own pace:

[Image Processing and Computer Vision\(CV\)](#)

[Matlab Self-Paced Online Courses:](#)

5.3.1 MatLab Image Processing Workflow



Import

You can import an image into memory and assign it to a variable in the MATLAB workspace for later manipulation, display, and export. Ending the line with a semicolon stops MATLAB from displaying the values stored in A, which is helpful because images typically contain millions of values. Image data stored in a variable A can be displayed using the `imshow` function.

You often want to view multiple images. For example, you might want to compare an image and a modified version of that image. You can display two images together using the `imshowpair` function. The "montage" option places the images A1 and A2 side by side, with A1 on the left and A2 on the right.

When you're done processing an image in MATLAB, you can save it to a file. The function `imwrite` exports data stored in the MATLAB workspace to an image file. The output file format is automatically identified based on the file extension.

```
A = imread("imgFile.jpg");
imshow(A)
A2 = ...;
imshowpair(A1, A2, "montage")
imwrite(imgData,"img.jpg")
```

Preprocess

Many image processing techniques are affected by image size. The images you'll be classifying all have the same size: 600 pixels tall and 400 pixels wide. You can find the size of an array by using the `size` function. The `size` function returns a vector. The first element in the vector is the number of rows in the array, which corresponds to the height of the image. The second element is the number of columns in the array, which corresponds to the width of the image.

Did the size of the array match the size of the image in pixels? Almost. You probably noticed the additional element in the vector returned by the `size` function: 3. The imported image is a color image, so that third element indicates the third dimension of the array, which stores the red, green, and blue color planes. You can access individual color planes of an image by indexing into the third dimension. For example, you can extract the green (second) color plane using the value 2 as the third index.

Most images use the unsigned 8-bit integer (`uint8`) data type, which stores integers from 0 to 255. Bright or brightly colored images contain pixel intensity values near 255 in one or more color planes. The red plane of this receipt image has some pretty bright areas. Do you think any elements of the red plane reach the maximum value of 255? You can find the largest value in an array using the `max` function. Using the "all" option finds the maximum across all values in the array. The brackets are required; they are placeholders for an unused input. Dark areas, like receipt text, contain values close to zero. Based on the image of the

red plane from Task 3, do you think the red plane has any elements with a value of 0? You can find the smallest value in an array using the `min` function.

Many common tasks can be completed more quickly using functions in Image Processing Toolbox. For example, to extract all three color planes of an image array, you can use `imsplit` instead of indexing into each plane individually. You can display all three color planes at once by using the `montage` function. Try using `imsplit` to extract the color planes of `I`. Then display all three color planes using `montage`.

```
sz = size(A) # height * width
Ig = I(:,:,2);
Amax = max(A,[],"all")
Amin = min(A,[],"all")
[R,G,B] = imsplit(A);
montage({R,G,B})
```

The algorithm you implement in this course is built around identifying text, so color is not necessary. Converting the images to grayscale eliminates color and helps the algorithm focus on the black-and-white patterns found in receipts. You can convert an image to grayscale using the `im2gray` function. If you need your converted grayscale images later, you can save them to a file. Try using `imwrite` to save `gs` to the file `gs.jpg`.

```
Ags = im2gray(A);
```

Why Convert to Grayscale?

- When loaded into memory, a grayscale image occupies a third of the space required for an RGB image.
- Because a grayscale image has a third of the data, it requires less computational power to process and can reduce computation time.
- A grayscale image is conceptually simpler than an RGB image, so developing an image processing algorithm can be more straightforward when working with grayscale.

Contrast and Intensity Histograms

If you are analyzing a set of images, normalizing the brightness can be an important preprocessing step, especially for identifying the black-and-white patterns of text in receipt images. An intensity histogram separates pixels into bins based on their intensity values. Dark images, for example, have many pixels binned¹ in the low end of the histogram. Bright images have many pixels binned at the high end of the histogram. The histogram often suggests where simple adjustments can be made to improve the definition of image features.

One of the essential identifying features of a receipt image is text. Receipt images should have good contrast so that the text stands out from the paper background. You can investigate the contrast in an image by viewing its intensity histogram using the `imhist` function. If the histogram shows pixels binned mainly at the high and low ends of the spectrum, then the receipt has good contrast. If not, it could probably benefit from a contrast adjustment. Increasing image contrast brightens brighter pixels and darkens darker pixels. You can use the `imadjust` function to adjust the contrast of a grayscale image automatically. You can, however, use the function `imlocalbrighten` to adjust the contrast of a color image.

```
imhist(I)
Iadj = imadjust(I);
Ibri = imlocalbrighten(I);
```

¹passive form of verb 'bin', means to store, to separate

Segment

The main feature of receipt images is dark text on a bright background. How can you leverage this feature? One method is to separate the text from the background. Separating an image into distinct parts is called segmenting an image.

Intensity Thresholding

You can create a binary black-and-white image from a grayscale image by thresholding its intensity values. Values below the cutoff are assigned the value 0, while values above are assigned the value 1. When applied to arrays, logical operators like `<` and `>` generate arrays of the same size that contain logical values 1 (true) or 0 (false). You can use logical operators to threshold the intensity values of a grayscale image, creating a binary image.

The ultimate goal is to create an algorithm that can process all of our images. Unfortunately, receipt images taken in different lighting conditions have varied contrast and require different thresholds. Manually identifying a threshold using the intensity histogram works but is impractical for thousands of images. To automate the threshold selection process, you can use the `imbinarize` function, which calculates the "best" threshold for the image. In `imbinarize`, you can designate whether the foreground is bright or dark by setting the "ForegroundPolarity" option.

```
B = g > 200;
gBinary = imbinarize(g);
BWadapt = imbinarize(gsAdj,"adaptive","ForegroundPolarity","dark");
```

Identifying Text Patterns

You've improved the visibility of the text in a binary image. Now how can you determine if the image is a receipt? The answer lies in the pattern of text in the image.

The rows of pixels that contain text have more 0 threshold values, and the rows between lines of text have more 1s. If you sum the values across each row, the rows with text have smaller sums than the rows without text. You can compute the sums across rows of an array by using the `sum` function operating along the second dimension of the array.

```
# second parameter,      dim = 1 sum down columns (default); dim = 2 sum across
    rows
rSum = sum(BW,2)
plot(rSum) # draw the sum
```

Preprocess and Postprocess

Segmentation can be improved in two ways: by preprocessing the image before binarizing and by postprocessing the binary image itself.

Spatial Filtering

Spatial filtering is a powerful technique to enhance images or their features.

- Noisy images can contain individual pixels with noticeably different intensity values than their neighbors.
- An averaging filter can smooth out these differences by replacing each pixel's value with the average of itself and neighboring pixels.
- A filter is a matrix that acts as a mask, creating a window around a pixel to perform operations.
- Applying a filter involves performing operations like weighted averages on the window, resulting in a new filtered image.
- Different filters serve different purposes, such as blurring edges or removing noise.

Images taken in low light often become noisy due to the increase in camera sensitivity required to capture the image. This noise can interfere with receipt identification by polluting regions in the binarized image. To reduce the impact of this noise on the binary image, you can preprocess the image with an averaging filter.

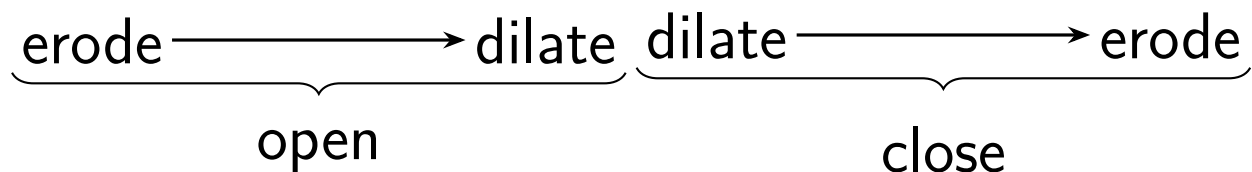
Use the `fspecial` function to create an n -by- n averaging filter. You can apply a filter `F` to an image `I` by using the `imfilter` function. If you look closely, you can see that filtering introduced a small, unwanted dark outline to parts of the binary image border. The reason is that the default setting of `imfilter` sets pixels outside the image to zero. To adjust this behavior, use the "replicate" option. Instead of zeros, the "replicate" option uses pixel intensity values on the image border for pixels outside the image.

```
F = fspecial("average", n)
Ifltr = imfilter(I,F);
Ifltr = imfilter(I,F,"replicate");
```

Isolate the Image Background

Receipt images that have a busy background are more difficult to classify because artifacts pollute the row sum. To mitigate this issue, you can isolate the background and then remove it by subtraction.

In a receipt image, the background is anything that is not text, so isolating the background can be interpreted as removing the text. One way to remove text from an image is to use morphological operations.



Morphological Operations

Morphological operations are sliding window operations that use a structuring element to determine which neighboring pixels to include for each calculation.

- You can change the size and shape of the structuring element to affect the output of morphological operations.
- Dilation takes the brightest pixel in the window, and erosion takes the darkest.
- Opening (erosion followed by dilation) emphasizes darker parts of an image and removes smaller bright areas.
- Closing (dilation followed by erosion) emphasizes brighter parts of an image and removes smaller dark areas.

In a receipt image, the background is anything that is not text. By using a structuring element larger than a typical letter, each "window" captures what's behind the text and not the text itself. You can create a structuring element by using the `strel` function.

The foreground (the text) is dark, and the background is bright. Closing the image emphasizes and connects the brighter background, removing the thin dark lettering. Larger background objects, like the thumb, remain. The final result is an image of the background without the text. To perform a closing operation on an image `I` with structuring element `SE`, use the `imclose` function.

Now that you've isolated the background `Ibg`, you can remove it from `gs`. In general, you can remove one image from another by subtracting it. Because `imclose` emphasizes brightness in an image, the background image `Ibg` has higher intensities than `gs`. Subtracting `Ibg` from `gs` would result in negative intensities.

The sequence of operations used here is very common: closing an image, then subtracting the original image from the closed image. It's known as the "bottom hat transform," and you can use the `imbothat` function in MATLAB to perform it.

```
SE = strel("diamond",5)
Iclosed = imclose(I,SE);
Isub = I2 - I1;
BWsub = ~ ib;
I = imbothat(I,Se);
```

Classification and Batch Processing

what is "Local Minima" in pictures in Matlab

A local minimum usually refers to a point in the image where a function (like brightness or intensity) is lower than its neighbors.

$$I(x_0, y_0) \leq I(x, y) \quad \text{for all neighbors } (x, y)$$

Matlab DataStore The `imageDatastore` function creates an image datastore for all the image files in a folder but won't load them into memory until they are requested. You can access the datastore's properties by using a period (.). The `readimage` function loads the `n`th image from an image datastore into the MATLAB workspace.

```
ds = imageDatastore("localFolder")
folders = ds.Folders
files = ds.Files
img = read(ds); # like c style, index might be loop from 0 to len-1
img = readimage(ds,n); # n>0
```

```
nFiles = numel(ds.Files);
isReceipt = false(1,nFiles);
for i=1:nFiles
    I = readimage(ds,i);
    isReceipt(i) = classifyImage(I);
end
receiptFiles = ds.Files(isReceipt)
montage(receiptFiles)
```

5.3.2 Conclusion

Table 5.1: Common MATLAB Image Processing Functions

Function	Description	Example
<code>imread</code>	Read an image into the workspace.	<code>imread("img001.png");</code>
<code>imshow</code>	Display an imported image.	<code>imshow(I)</code>
<code>imshowpair</code>	Display two images as an overlay or montage.	<code>imshowpair(I1,I2,"montage")</code>
<code>montage</code>	Display multiple images next to each other.	<code>montage({I1, I2, I3})</code>
<code>im2gray</code>	Convert an RGB image to grayscale.	<code>im2gray(I)</code>
<code>imhist</code>	Display a histogram of a grayscale image.	<code>imhist(gs)</code>
<code>imadjust</code>	Adjust the contrast of an image.	<code>imadjust(gs)</code>
<code>imbinarize</code>	Create a binary image from a grayscale image.	<code>imbinarize(gs)</code>
<code>imresize</code>	Resize an image to half of its origin.	<code>imresize(I, 0.5)</code>
<code>imcrop</code>	Crop an image to get smaller size.	<code>imcrop(I,[x,y,w,h])</code>
<code>fspecial</code>	Create a filter from predefined filter types.	<code>fspecial("disk",5)</code>
<code>imfilter</code>	Apply filter H to an image.	<code>imfilter(I,H)</code>
<code>strel</code>	Create a morphological structuring element.	<code>strel("rectangle",[5 5])</code>
<code>imclose</code>	Morphological closing using SE.	<code>imclose(I,SE)</code>
<code>imopen</code>	Morphological opening using SE.	<code>imopen(I,SE)</code>
<code>imageDatastore</code>	Create datastore for image data.	<code>imageDatastore("myFolder")</code>
<code>readimage</code>	Read the <code>n</code> th image from datastore.	<code>readimage(ds,n)</code>

Note: Functions are from MATLAB Image Processing Toolbox.

5.3.3 Demo

```
# matlab online
turtleVideo = VideoReader("turtles.avi")

#Remember to use a semicolon (;) so the values in frame are not displayed.
frame=readFrame(turtleVideo);
imshow(frame)
t=turtleVideo.CurrentTime
frame2=readFrame(turtleVideo);
t2=turtleVideo.CurrentTime
imshowpair(frame,frame2)

frameMid=read(turtleVideo,50);
imshow(frameMid)
tMid=turtleVideo.CurrentTime

# like C, continue from frame 50,
frameNext=readFrame(turtleVideo);
imshow(frameNext)
tNext=turtleVideo.CurrentTime

# last frame;
frameLast=read(turtleVideo,Inf);
imshow(frameLast)
tLast=turtleVideo.CurrentTime

# rewind
turtleVideo.CurrentTime=0
```

5.4 Lecture 3

Table 5.2: SWOT

internal factors		external factors	
Strengths	Weaknesses	Opportunities	Threads
things you do well	limitations or disadvantages	growth or improvement	risks or challenges
- brand reputation - skilled employees - advanced technology - good financial res.	- lack of experience - outdated technology - poor marketing - limited budget	- new market demand - tech. breakthroughs - government support - increasing popularity of AI	- strong competitors - economic downturn - new regulations - rapid tech. changes

SWOT is a simple framework used in strategic planning to analyze a situation, project, company, or idea.

blue-sky research

Also called **pure research** or **basic research** refers to research done without a specific practical goal or immediate commercial application in mind. The main purpose is to expand knowledge and understanding, even if the usefulness is not clear at the time.

5.5 Other Learning Resources

AI Basketball Analysis

[AI-basketball-analysis-google-colab.ipynb](#)

Chapter 6

Mathematics

6.1 Norm

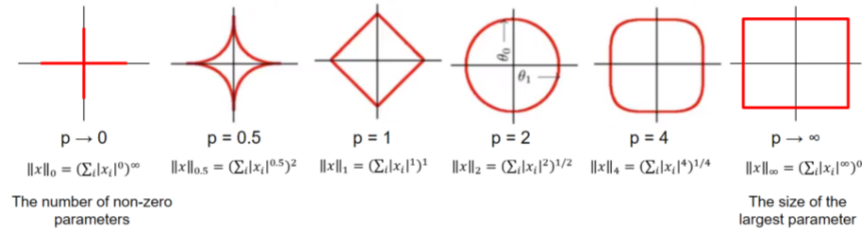


Figure 6.1: L_n norm illustrated in a 2D space.

6.2 Harmonic Mean

The Harmonic Mean is a specialized type of average that is used primarily when dealing with rates, ratios, or speed. It is one of the three "Pythagorean means," alongside the Arithmetic Mean and the Geometric Mean. For a set of n positive numbers $x_1, x_2, \dots, x_n$, the Harmonic Mean (H) is defined as the reciprocal of the arithmetic mean of the reciprocals:

$$H = \frac{n}{\sum_{i=1}^n \frac{1}{x_i}} \quad (6.1)$$

Unlike the arithmetic mean, the harmonic mean is heavily influenced by **small values**. If either Precision or Recall is very low, the F1-Score drops significantly. This forces the model to perform well on both metrics rather than relying on one high score to mask a failing one.

6.3 Marginal Probability

In probability theory, marginal probability is the probability of one variable occurring regardless of the values of other variables. Mathematically, it is obtained by summing or integrating over the other variables in a joint distribution. For example, if we have a joint probability $P(X, Y)$, the marginal probability of X is:

$$P(X) = \sum_y P(X, y) \quad (6.2)$$

Table 6.1: Comparison of Pythagorean Means

Mean Type	Formula	Best Used For
Arithmetic Mean	$\frac{1}{n} \sum_{i=1}^n x_i$	General data, linear sets
Geometric Mean	$\left(\prod_{i=1}^n x_i \right)^{1/n}$	Growth rates, percentages
Harmonic Mean	$\frac{n}{\sum_{i=1}^n \frac{1}{x_i}}$	Rates, ratios, speed, F1-score

Note: For any set of positive numbers, the relationship Arithmetic $\geq$ Geometric $\geq$ Harmonic always holds.

The above formula is for discrete variables, as for continuous ones, we shall use calculus to sum others up:

$$P(X) = \int P(X, y) \, dy \quad (6.3)$$

6.4 Conditional Probability

Conditional probability describes the probability of an event given that another event has already occurred.

$$P(A \mid B) = \frac{P(A \cap B)}{P(B)} \quad (6.4)$$

Table 6.2: Basic probability concepts

Concept	Meaning	Description
Joint probability	$P(A \cap B)$	probability that A and B occur together
Marginal probability	$P(A)$	probability of A
Conditional probability	$P(A \mid B)$	probability that A occurs given B has occurred
Independence	$P(A \mid B) = P(A)$	

6.5 Distributions

6.5.1 Bernoulli Distribution

The Bernoulli distribution is one of the simplest probability distributions in statistics. It models a random experiment that has exactly two possible outcomes, usually labeled as: 1 for "success" and 0 for "failure".

Key Features:

- **Single Trial:** It describes the outcome of one trial (like flipping a coin once).
- **Probability Parameter:** It has a parameter p , which is the probability of success (outcome 1). Then the probability of failure (outcome 0) is $1 - p$

Probability Mass Function (PMF):

$$P(X = x) = \begin{cases} p & \text{if } x = 1 \\ 1 - p & \text{if } x = 0 \end{cases}$$

where $0 \leq p \leq 1$

6.5.2 Binomial Distribution

The Binomial distribution generalizes the Bernoulli distribution to multiple independent trials.

Key Idea:

- You perform n independent Bernoulli trials (each trial has two outcomes: success or failure).
- Each trial has the same probability of success p .
- You're interested in the total number of successes in those n trials.

Parameters

- n = number of trials
- p = probability of success in a single trial

Probability Mass Function (PMF)

The probability of getting exactly k successes in n trials is:

$$P(X = k) = \binom{n}{k} p^k (1 - p)^{n-k}, \quad k = 0, 1, 2, \dots, n$$

Mean and Variance

- Mean: $\mu = E[X] = np$
- Variance: $\sigma^2 = np(1 - p)$

6.5.3 Poisson Distribution

The Poisson distribution is a *discrete* probability distribution that expresses the probability of a given number of events occurring in a fixed interval of time or space. These events must occur with a known constant mean rate and independently of the time since the last event. The probability of observing exactly k events in a specific interval is given by:

$$P(X = k) = \frac{\lambda^k e^{-\lambda}}{k!}$$

- λ : The average number of events in the given interval (the mean).
- k : The actual number of events (an integer 0,1,2,...).
- e : Euler's number (approximately 2.71828).
- $k!$: The factorial of k .

Key Assumptions

For a process to be modeled by a Poisson distribution, four conditions generally apply:

- Discrete Events: The events are countable (e.g., 0, 1, 2, 3...).
- Independence: The occurrence of one event does not affect the probability of the next one occurring.
- Constant Rate: The average rate (λ) at which events occur is constant over the interval.
- Single Event: Two events cannot occur at the exact same instant.

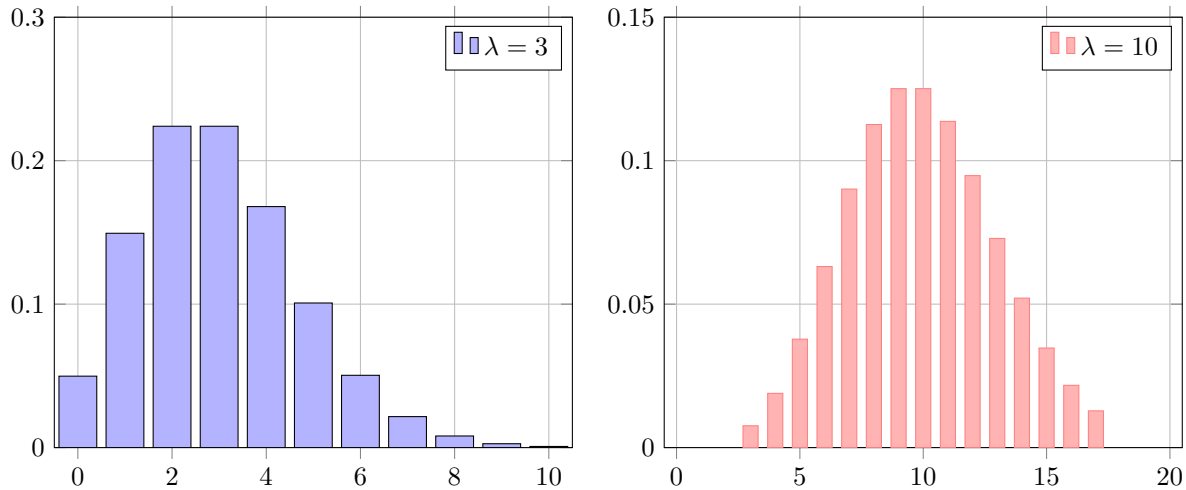


Figure 6.2: Poisson distribution with different λ . X-axis demonstrates number of events (k), while y-axis represents Probability of event ($P(X = k)$). As λ gets larger, the Poisson distribution begins to look more like a Normal distribution (right figure).

Key Properties

- **Mean and Variance:** A unique property of the Poisson distribution is that its mean is equal to its variance. **Both are equal to λ .** ($E[X] = Var(X) = \lambda$)
- **As λ Increases:** As the average rate (λ) gets larger, the Poisson distribution begins to look more like a Normal (Gaussian) distribution (Figure 6.2).

6.5.4 Normal Distribution

In statistics, the Normal Distribution (often called the Gaussian Distribution or the "Bell Curve") is a probability distribution that is symmetric about the mean. It shows that data near the mean are more frequent in occurrence than data far from the mean.

$$p(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2} \quad (6.5)$$

It is perhaps the most important concept in statistics because of the **Central Limit Theorem** (Section 6.6), which states that the sum of many independent random variables tends toward a normal distribution, regardless of the original distribution shape.

Key Characteristics

- **Symmetry:** The left and right halves are mirror images.
- **Measures of Central Tendency:** The mean, median, and mode are all exactly the same and located at the center (peak) of the curve.
- **Total Area:** The total area under the curve is always equal to 1 (or 100%), representing the total probability of all possible outcomes.

6.5.5 Exponential Distribution

The Exponential distribution is a continuous probability distribution that describes the time between events in a Poisson point processessentially, the time you wait until the next event occurs. While the Poisson distribution counts how many events happen in a fixed interval, the Exponential distribution measures the

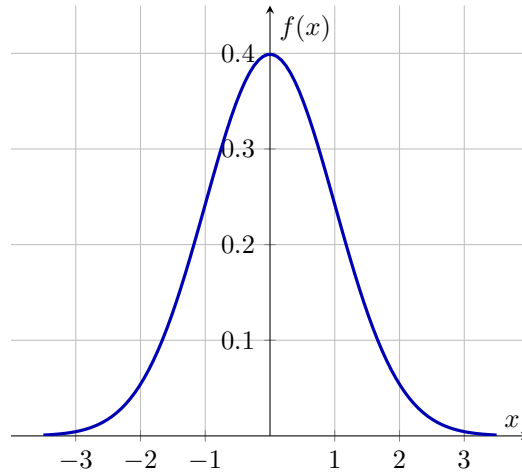


Figure 6.3: Standard Normal distribution ($\mu = 0, \sigma = 1$)

duration between those events. For a random variable X (the waiting time), the probability density is given by:

$$p(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

- λ (Rate parameter): The average number of events per unit time.
- x : The time elapsed until the next event.

6.6 Central Limit Theorem

6.7 Pearson chi-square

The Pearson chi-square (6.6) is a fundamental concept in statistics, especially in testing relationships between categorical variables. Lets break it down clearly and step by step.

$$\chi^2 = \sum \frac{(O - E)^2}{E} \quad (6.6)$$

where:

O = observed frequency

E = expected frequency under independence

6.7.1 Step 1/4: Gethering Data

Suppose we surveyed 100 people and collected the data (Table 6.3), we want to know: Is there a relationship between drinking coffee and insomnia?

6.7.2 Step 2/4: Calculate Expected Counts E

The expected count formula for each cell is:

$$E = \frac{(\text{row total}) \times (\text{column total})}{\text{grand total}}$$

- Drinks Coffee & Insomnia: $E=18$

Table 6.3: Demo data to demonstrates Pearson chi-square

	Insomnia	No Insomnia	Total
Drinks Coffee	25	15	40
No Coffee	20	40	60
Total	45	55	100

- Drinks Coffee & No Insomnia: $E=22$
- No Coffee & Insomnia: $E=27$
- No Coffee & No Insomnia: $E=33$

6.7.3 Step 3/4: Calculate Pearson chi-square result

- Drinks Coffee & Insomnia: $\frac{(O-E)^2}{E} = \frac{(25-18)^2}{18} \approx 2.722$
- Drinks Coffee & No Insomnia: ≈ 2.227
- No Coffee & Insomnia: ≈ 1.815
- No Coffee & No Insomnia: ≈ 1.485
- **Sum up:** $\chi^2 \approx 2.722 + 2.227 + 1.815 + 1.485 = 8.249$

6.7.4 Step 4/4: Calculate Dispersion Parameter and Interpret

We now know $\chi^2 \approx 8.25$ and $df = (\text{rows}-1) \times (\text{columns}-1) = (2-1)(2-1) = 1$ (6.7), We got p-value < 0.01 . This suggests a **significant association** between drinking coffee and insomnia.

$$\phi = \frac{\chi^2}{\text{degree of freedom}} \quad (6.7)$$

6.8 Covariance

In simple terms, covariance is a measure of how two variables change together. If one variable goes up, does the other tend to go up as well, or does it go down? While it tells you the direction of a relationship, it doesn't tell you the strength (that's what correlation is for).

$$\text{cov}(X, Y) = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{n - 1} \quad (6.8)$$

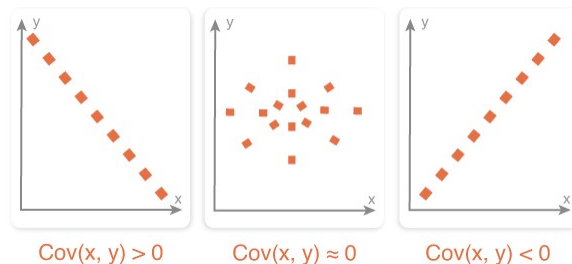


Figure 6.4: Result of Covariance, positive value indicates same change trend, while negative means one becomes higher with another declines. From: [mathmonks](#)

- **Positive Covariance:** Both variables move in the same direction. When X increases, Y tends to increase (e.g., height and weight).
- **Negative Covariance:** The variables move in opposite directions. When X increases, Y tends to decrease (e.g., time spent partying and exam scores).

- **Zero Covariance:** There is no linear relationship between the variables. They move independently of each other.

Table 6.4: Comparison: Covariance Matrix vs. Correlation Matrix

Feature	Covariance Matrix	Correlation Matrix
Description	Measures the absolute linear relationship between variables. It is highly sensitive to the units of measurement (e.g., if you measure distance in meters versus millimeters, the covariance values will change drastically).	Measures the standardized linear relationship. It is essentially a "rescaled" version of the covariance matrix where each variable is divided by its standard deviation.
Scale Sensitive	Yes (Depends on units)	No (Dimensionless)
Range	$(-\infty, +\infty)$	$[-1, +1]$
Interpretation	Magnitude and Direction	Strength and Direction
Use Case	Same measurement units	Different measurement units

Note: The correlation matrix is essentially the covariance matrix normalized by the standard deviations of the variables.

6.9 Softmax

The softmax function is one of the most important pieces in classification models (especially neural networks like CNNs). It turns raw model outputs into probabilities.

6.9.1 What softmax does (intuition)

A neural network usually outputs a vector of arbitrary real numbers (called logits), e.g.: $[2.0, 1.0, 0.1]$. These are not probabilities: i) They don't sum to 1; ii) They can be negative or large. Softmax converts them into probabilities: i) All values between 0 and 1; ii) Sum = 1; iii) Higher input means higher probability.

$$\sigma(z_i) = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}} \quad (6.9)$$

Example

Input logits: $[2.0, 1.0, 0.1]$
 Exponentiate: $[e^2, e^1, e^{0.1}] \approx [7.39, 2.72, 1.11]$
 Sum: $7.39 + 2.72 + 1.11 = 11.22$
 Normalize: $[0.66, 0.24, 0.10]$
 Class 1: 66%, Class 2: 24%, Class 3: 10%

6.9.2 Key properties

- Probability distribution: outputs sum to 1
- Order-preserving: largest logit means largest probability
- Smooth & differentiable: good for gradient-based training
- Amplifies differences: bigger logits dominate more

6.9.3 Why exponentials? (important insight)

Using e^x :

- Ensures outputs are positive
- Makes differences more pronounced: A small increase in logit noticeable probability jump

6.9.4 Numerical stability trick (very important in practice)

Direct softmax can overflow if logits are large. So we compute:

$$\sigma(z_i) = \frac{e^{z_i - \max(z)}}{\sum_{j=1}^K e^{z_j - \max(z)}} \quad (6.10)$$

This does not change the result, but avoids huge numbers.